

Automated Verification of Consistency in Zero-Knowledge Proof Circuits

Jon Stephens^{1,2} , Shankara Pailoor² , Isil Dillig^{1,2} 

¹The University of Texas at Austin. {jon, isil}@cs.utexas.edu

²Veridise Inc. {jon, shankara, isil}@veridise.com

Abstract. Circuit languages like Circom and Gnark have become essential tools for programmable zero-knowledge cryptography, allowing developers to build privacy-preserving applications. These domain-specific languages (DSLs) encode both the computation to be verified (as a *witness generator*) and the corresponding *arithmetic circuits*, from which the prover and verifier can be automatically generated. However, for these programs to be correct, the witness generator and the arithmetic circuit need to be mutually consistent in a certain technical sense, and inconsistencies can result in security vulnerabilities. This paper formalizes the consistency requirement for circuit DSLs and proposes the first automated technique for verifying it. We evaluate the method on hundreds of real-world circuits, demonstrating its utility for both automated verification and uncovering errors that existing tools are unable to detect.

1 Introduction

Zero-knowledge (ZK) proofs are cryptographic protocols that allow a prover to convince a verifier of the truth of a statement without disclosing confidential information. These protocols have become crucial building blocks in implementing privacy-preserving applications. Recent advancements in cryptography have introduced a powerful class of *succinct* zero-knowledge proofs [1, 2] that enable verification of complex computations while keeping both proof sizes and verification times minimal. This innovation has enabled a wide range of applications, including anonymous whistleblowing [3], image authentication [4], private digital transactions [5], and ensuring computational integrity in blockchains [6].

However, successfully integrating zero-knowledge proofs into an application requires developers to create two interdependent artifacts: (1) a set of constraints, formulated as polynomial equations over a finite field and (2) a *witness generator* that produces satisfying values for these constraints. Crucially, a fundamental assumption about these artifacts is that the constraints should only accept values that are produced by the witness generator. Failure to meet this requirement can lead to severe security vulnerabilities, allowing true statements to become unprovable or, even more seriously, false statements to be incorrectly verified.

To aid developers in defining these two artifacts, several domain-specific languages (DSLs) have been developed. These “circuit DSLs”, such as Circom and Gnark, streamline the process of maintaining consistency between constraints

and witness generators. For example, several DSLs include constructs that allow developers to define constraints and assignments simultaneously, reducing the risk of errors. Despite these tools, inconsistencies remain a major challenge, with over 95% of known ZK vulnerabilities arising from such issues [7].

This paper aims to address these challenges by introducing a formal definition of consistency between constraints and witness computation and presenting the first automated algorithm to verify this notion of consistency in circuit DSLs. Informally, a circuit is *consistent* if its constraints accept only those witnesses generated by the witness generator. While prior work has focused on detecting specific instances of inconsistency (such as non-determinism) [8–11], none have provided an automated solution for verifying the stronger notion consistency formalized in this paper.

From a technical perspective, the consistency verification problem poses two key challenges. First, it requires proving a relational property between a witness generator and an arithmetic circuit, which oftentimes defines non-linear finite field equations that tend to challenge SMT solvers. Second, both of these artifacts are typically derived from *templates* containing arrays and complex loops, necessitating the inference of complex quantified loop invariants.

This paper tackles the consistency verification problem through a two-step approach. First, given a *product program* that simultaneously performs constraint generation and witness construction, our approach uses lightweight static reasoning to infer pair-wise equivalences between array elements and simplifies the initial product program as much as possible. In the subsequent verification step, our method infers more complicated quantified invariants over arrays to discharge the verification conditions needed for proving equivalence.

We have implemented our method in a tool called ZEQUAL, which verifies consistency of circuit templates written in Circom, a widely-used circuit DSL and the target language of several other ZK circuit verifiers and bug-finding tools in prior literature [8–10, 12]. We evaluate ZEQUAL on hundreds of real-world benchmarks taken from popular projects and show that ZEQUAL can verify consistency in a majority of these templates. We also demonstrate that ZEQUAL is capable of finding bugs that cannot be detected using prior methods.

To summarize, contributions of this paper include: (1) a formal definition of the *consistency verification problem* for ZK circuits; (2) first automated technique for verifying consistency of ZK circuit templates; (3) evaluation of the method on several hundred real-world circuit templates.

2 Background

Zero-Knowledge Proofs (ZKPs) are protocols that allow a prover to demonstrate the truth of a statement to a verifier without revealing any additional information beyond the statement’s validity. Traditionally, ZKPs were developed for specific statements such as the three-coloring problem or graph isomorphism; however, new advancements in Programmable ZKP have enabled greater flexi-

```

1 template MultiMux() {
2   signal input x[5]; // Constants
3   signal input y[5]; // Constants
4   signal input s; // Selector
5   signal output out[5];
6   for (var i=0; i<5; i++) {
7     out[i] <== (x[i] - y[i])*s + x[i];
8   }
9 }

1 template IsZero() {
2   signal input in;
3   signal output out;
4   signal inv;
5   inv <-- in!=0 ? 1/in : 0;
6   out <== -in*inv +1;
7   in*out == 0; }

```

Fig. 2: Examples of Circom templates.

bility. For example, proof frameworks like zk-SNARKs [1] or zk-STARKs [2] can generate verifiable proofs that $P(x) = y$ for some computation P and private input x . A key property of programmable ZKPs is that a prover and verifier can be automatically generated from a set of polynomial equations over a finite field $\mathbb{F}_p$. The variables in these equations are referred to as signals, and addition and multiplication are performed modulo a large prime p . The details of how the prover and verifier are generated are beyond the scope of this paper, but we refer the interested reader to [1, 13, 14] for additional background.

Once the prover and verifier are set up, proofs are generated and verified as illustrated in Figure 1. The process begins with the witness generator, a program that produces a *witness*—a mapping of signals to values that satisfy the constraints. Conceptually, the witness generator executes the computation P for which the user seeks to prove correctness. The prover then uses this witness, along with the constraints, to generate a proof, which is sent to the verifier. Crucially, the proof conceals the values of all private signals. The verifier receives the proof and only the *public* values, verifying its correctness or rejecting it. Notably, witnesses do not need to be generated solely by running the witness generator; any satisfying assignment to the constraints can produce a verifiable proof. As a result, it is essential that the constraints accurately reflect the intended computation and align with the witness generator. Inconsistencies between these artifacts could lead to invalid proofs being accepted or valid computations failing to produce verifiable proofs. Indeed, most security vulnerabilities in real-world implementations of these protocols stem from such inconsistencies [7].

Circom DSL. Manually encoding a computation as a set of constraints is a time-consuming and error-prone task. To address these challenges, several domain-specific languages have been developed [15–17], with Circom being one of the most popular [16]. Circom allows developers to specify the witness generation logic and constraints in a unified language (with concepts similar to a hardware description language (HDL)), and the compiler automatically generates the witness generator along with the constraints. Tools like SnarkJS [18] can then be used to generate the prover and verifier from these constraints.

A Circom program consists of a list of *templates* with one template distinguished as the entry point. A template is similar to a sub-circuit module in a

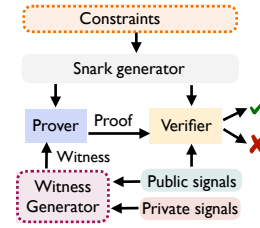


Fig. 1: ZK workflow.

```

1 template Divide() {
2   signal input numerator, denominator;
3   signal output remainder, quotient;
4   isZero := IsZero();
5   isZero.in <== denominator;
6   isZero.out == 0; // ensure denominator is non-zero
7   remainder <-- numerator % denominator;
8   quotient <-- numerator \ denominator;
9   numerator == denominator * quotient + remainder;
10 }

1 template Reward() {
2
3   signal input inp;
4   signal output out;
5   var gwei = 10 ** 6;
6   out <-- inp \ gwei;
7   out * gwei == inp;
8 }
9

```

Fig. 3: Non-determinism bug on left; inconsistency bug on right.

HDL as it defines a reusable block of code that is instantiated upon invocation by connecting the declared input and output signals within the caller. All output signals in Circom are public and other signals are private by default unless otherwise specified. Witness generation is expressed using the `<--` and `-->` operators for signal assignment, while the `==` operator expresses constraints. Additionally, the `==>` and `<==` operators can be used to combine assignment and constraint declaration when the left and right hand sides are polynomials, helping developers maintain consistency between computation and constraints. For example, the program on the left in Figure 2 only performs addition and multiplication over signals—operations that are directly expressible as polynomials over finite fields. As such, it is able to exclusively use the `<==` operator, keeping the witness generation and constraints consistent. In contrast, the program in Figure 2(b) does not use the `<==` operator because comparisons cannot be expressed directly as polynomials. Instead, the corresponding constraints utilize an intermediate signal, `inv`, relying on the field axiom that every nonzero element has a multiplicative inverse.

Circuit Bugs. Languages like Circom help developers maintain consistency between witness generation and constraints, but implementations often contain bugs due to discrepancies. A common issue, known as “Unintended Nondeterminism” or “Underconstrained Circuits”, arises when constraints $C(in, out)$ allow multiple outputs o and o' for the same input i . The program on the left of Figure 3 provides an example of a nondeterminism bug: the template takes a numerator and denominator as inputs and returns the quotient and remainder. The witness generation code performs this logic directly using the integer division operator and the modulo operator `%`. However, the constraints only assert that `quotient * numerator + remainder = numerator`, lacking a crucial check that the remainder is *less than* the denominator when interpreted as integers. Without this check, there are multiple solutions for a given input.

However, not all discrepancies manifest themselves as nondeterminism bugs: constraints can be deterministic, yet their satisfying assignments may still diverge from the witness values, as illustrated on the right of Figure 3¹. This program takes an input currency amount `inp`, and the witness generator returns a reward `out` by dividing `inp` by a constant `gwei` using integer division. Here, the computation and constraints only align when `inp` is a multiple of `gwei` but

¹This example was taken from [9] and is a simplified version of a bug found in a real-world closed-source project.

```

1 template DecomposeProduct(N) {
2   signal input u8_xs[N], u8_ys[N];
3   signal u16s[N];
4   signal output high[N], low[N];
5   component bits_low[N], bits_high[N];
6
7   for (var i = 0; i < N; i++) {
8     u16s[i] <== u8_xs[i] * u8_ys[i];
9     low[i] <-- u16s[i] % 0x100;
10    bits_low[i] = Num2Bits(8);
11    bits_low[i].in <== low[i];
12    high[i] <-- (u16s[i] \ 0x100) % 0x100;
13    bits_high[i] = Num2Bits(8);
14    bits_high[i].in <== high[i];
15    u16s[i] == low[i] + 0x100 * high[i];
16  }
17}

```

Fig. 4: Code for motivating example

```

1 template DecomposeProduct_Expanded(N) {
2   signal input u8_xs[N], u8_ys[N];
3   signal u16s_w[N], u16s_c[N];
4   signal output high_w[N], high_c[N];
5   signal output low_w[N], low_c[N];
6   component bits_low[N], bits_high[N];
7
8   for (var i = 0; i < N; i++) {
9     u16s_w[i] <-- u8_xs[i] * u8_ys[i];
10    u16s_c[i] == u8_xs[i] * u8_ys[i];
11    low_w[i] <-- u16s_w[i] % 0x100;
12    bits_low[i] = Num2Bits(8);
13    bits_low[i].in_w <-- low_w[i];
14    bits_low[i].in_c == low_c[i];
15    high_w[i] <-- (u16s_w[i] \ 0x100) % 0x100;
16    bits_high[i] = Num2Bits(8);
17    bits_high[i].in_w <-- high_w[i];
18    bits_high[i].in_c == high_c[i];
19    u16s_c[i] == low_c[i] + 0x100 * high_c[i];
20  }
21}

```

Fig. 5: Product program

diverge in all other cases. For instance, if `inp` is 1, the witness will assign `out` to 0, while the satisfying assignment for the constraints sets `out` to gwei^{-1} . This results in the output (as defined by the constraints) being even larger than the input, which could allow gaining more reward than intended.

3 Motivating Example

Figure 4 presents a Circom template which takes two arrays, `u8_xs` and `u8_ys` of length N , each containing field elements whose integer representation is an unsigned 8-bit integer. The template computes the element-wise product, capturing the result as 16-bit unsigned integers in `u16s`. Each entry in `u16s` is then decomposed into two output arrays, `low` and `high`, which store the lower and higher 8 bits, respectively.

At first glance, it is not immediately obvious why the witness generator and the constraints expressed by this program align with each other. For instance, the witness generator computes the low and high bytes in a standard way using integer division and the modulo operator (lines 9, 12), whereas the constraints enforce this decomposition at the bit level, validating each byte component individually through binary representations (`Num2Bits`) (lines 10, 13). Given the nontrivial differences between the witness generator and the constraints, establishing their equivalence over all possible template instantiations is challenging. In the following discussion, we explain the key ideas that enable ZEQUAL to perform verification successfully, along with the observations that inspired these ideas.

Observation #1: *Many circuit DSLs provide a “pseudo-product-program” with good alignment.* A common approach to verifying equivalence is to reduce the relational verification task to checking assertions in a *product program* [19]. A crucial step in this reduction is establishing a suitable *alignment* — a mapping that pairs corresponding operations between the two programs. In general-

purpose languages, achieving this alignment can be difficult because the program structures may differ significantly.

Fortunately, circuit DSLs like Circom, Gnark, and Zirgen implicitly define a “pseudo-product-program” with good alignment where corresponding operations between the witness and constraint components are grouped within the same basic block. We call this a *pseudo-product* because it uses the *same variables* to represent signals in both the constraints and the witness generator even when they may *not* be equivalent. Hence, in order to instrument the program with suitable assertions that ensure equivalence, our method modifies the Circom program by introducing two copies s_c, s_w for each (non-input) signal. For instance, as shown in Figure 5, `high[N]` is replaced with two arrays `high_w[N]` and `high_c[N]` denoting the constraint and witness generation components respectively.

Observation #2: *Lightweight static analysis can help infer pair-wise equivalence between signals, simplifying the verification task.* As mentioned in Section 2, circuit DSLs often include constructs that generate equivalent witness generation logic and constraints from a single statement. For instance, in Circom, the statement `x <== e` serves as shorthand for both the assignment `x <- e` and the constraint `x === e`. Other circuit languages also have similar features.²

This duality between witness generator logic and constraints is commonly leveraged by circuit developers to maintain consistency across the two artifacts. This makes it feasible to infer pair-wise equality between the two copies s_c, s_w of a signal s through *lightweight static analysis* of the product program. For example, at line 9 of Figure 5, `u16s_w[i]` is assigned to some expression e and line 10 includes constraint `u16s_c[i] === e` using the same expression e . Because e only involves input signals, we can conclude that `u16s_w[i]` and `u16s_c[i]` are pair-wise equivalent and then propagate this information to potentially prove equivalence of more signals. Finally, for any pair of signals proven equivalent, we can simplify the product program by deleting assignments and replacing the two copies of a signal with a single variable. In this example, we can delete the assignment at line 9 and replace `u16s_w` and `u16s_c` with a single variable `u16s`.

Observation #3: *Invariants needed for verification often have a particular shape.* After simplifying the product program, our approach instruments it with assumptions and assertions (see Figure 6) and tries to discharge the latter. However, because circuit programs often contain loops, ZEQUAL needs to synthesize quantified array invariants. While this is a challenging problem in general, most required invariants involve predicates of the form $\forall i. \phi(i) \rightarrow s_w[f(i)] = s_c[f(i)]$ where ϕ is a guard predicate over the loop counters and f is an arithmetic function. Our method generates candidate invariants of this shape through static analysis and computes the strongest conjunction over this universe of predicates

²For example, in Gnark and Cairo, every operation, *by default*, is translated both into a statement (in the witness generator) and a field equation; when this is not possible, the developer needs to employ so-called “hints”. Similarly, in Zirgen, computation performed in a component generates equivalent witness generator logic and constraints while computation performed in an extern is only performed in the witness generator.

```

1 template DecomposeProduct_Instrumented(N) {
2   signal input u8_xs[N], u8_ys[N];
3   signal u16s[N];
4   signal output high_w[N], high_c[N];
5   signal output low_w[N], low_c[N];
6   component bits_low[N], bits_high[N];
7
8   for (var i = 0; i < N; i++) {
9     assume(u16s[i] = u8_xs[i]*u8_ys[i]);
10    low_w[i] <-- u16s_w[i] % 0x100;
11    bits_low[i] = Num2Bits(8);
12    bits_low[i].in_w <-- low_w[i];
13    assume(bits_low[i].in_c = low_c[i]);
14    assert(bits_low[i].in_w = low_w[i]);
15    high_w[i] <-- (u16s_w[i] \ 0x100) % 0x100;
16    bits_high[i] = Num2Bits(8);
17    bits_high[i].in_w <-- high_w[i];
18    assume(bits_high[i].in_c = high_c[i]);
19    assert(bits_high[i].in_w = high_w[i]);
20    assume(u16s[i] = low_c[i] + 0x100 * high_c[i]);
21    assert(u16s[i] = low_w[i] + 0x100 * high_w[i]);
22  }
23  assert(low_w = low_c && high_w = high_c);
24 }

```

Fig. 6: Instrumented product

through monomial predicate abstraction [20]. For our example, the following invariant is sufficient for successful verification:

$$\forall x. 0 \leq x < i \leq N \rightarrow (\text{low_w}[x] = \text{low_c}[x] \wedge \text{high_w}[x] = \text{high_c}[x])$$

4 Problem Statement

In this paper, we adopt a formalization that captures commonalities between most circuit DSLs, such as Circom [16], Gnark [15], Zircen [21], and Zokrates [22].

Definition 1 (Circuit program). A ZK circuit program P is a tuple $(\Gamma, \mathbb{F}_p, \Phi, \omega)$ where:

- Γ is a set of signal declarations $s : \tau$, where $\tau \in \{\text{In}, \text{Out}, \text{Temp}\}$;
- $\mathbb{F}_p$ is the prime field of the circuit – i.e., $s \in \mathbb{F}_p$ for any signal s ;
- Constraint Φ is a formula in the theory of finite fields [23]. The precise language for such formulas is shown on the left side of Figure 7.
- Witness computation ω is a loop-free program over signals in Γ . The language for expressing witness computation is shown on the right side of Figure 7.

Given a program $P = (\Gamma, \mathbb{F}_p, \Phi, \omega)$, we write P_Φ, P_ω to denote Φ and ω respectively, and, for a signal s , $\Gamma(s)$ denotes the type of s . We refer to signals of type In, Out, and Temp as input, output, and intermediate signals respectively. Witness computation ω can *only* perform assignments to output and intermediate signals but never to inputs. Furthermore, ω can never perform multiple assignments to the same signal within the same execution.

The semantics of ZK circuit programs are defined over *signal valuations* σ that map signals to concrete values. The output of a program is a tuple (b, σ') where b is a boolean and σ' is the output valuation. Given a program $P =$

$$\begin{array}{ll}
\text{Constraints } \Phi ::= \Phi_1 \wedge \Phi_2 \mid P_1 == P_2 & \text{Program } \omega ::= s \leftarrow E \mid \omega_1; \omega_2 \\
& \mid \text{if}(C) \omega_1 \text{ else } \omega_2 \\
\text{Polynomial } P ::= P_1 + P_2 & \text{Conditional } C ::= E_1 \otimes E_2 \mid \neg C \\
& \mid P_1 * P_2 \mid s \mid c & \text{Expression } E ::= E_1 \oplus E_2 \mid s \mid c \mid E^{-1}
\end{array}$$

Fig. 7: (Left) Arithmetic circuit DSL. P is a polynomial equation over a finite field $\mathbb{F}_p$. (Right) Witness generation DSL over $\mathbb{F}_p$. E^{-1} computes the multiplicative inverse of E , and $\oplus, \otimes$ denote arithmetic and logical operators respectively.

($\Gamma, \mathbb{F}_p, \Phi, \omega$) we write $(\sigma, P) \Downarrow (b, \sigma')$ iff $\llbracket \Phi \rrbracket(\sigma)$ evaluates to b and executing ω on σ yields new environment σ' . Next, we define a notion of *agreement* between the witness computation and constraint in ZK circuit programs.

Definition 2 (Agreement). Let $P = (\Gamma, \mathbb{F}_p, \Phi, \omega)$ be a program and let σ be a signal valuation such that $(\sigma, P) \Downarrow (b, \sigma')$. We say that constraint Φ and witness computation ω agree on σ , denoted $\sigma \vdash \Phi \sim \omega$, iff the following holds:

$$b \rightarrow (\llbracket \Phi \rrbracket(\sigma') \wedge \forall (s : \text{Out}) \in \Gamma. \sigma(s) = \sigma'(s)) \quad (1)$$

According to this definition, if the constraint Φ “accepts” some valuation σ (meaning b is true), then (1) the output valuation σ' produced by the witness generator should also be accepted by Φ , and (2) the two valuations σ and σ' should agree on the values of all output signals. In other words, if a given witness σ with inputs I is accepted by the constraints Φ , then the witness generator ω should also be able to produce a satisfying witness σ' with the same inputs I such that the output signals in σ and σ' match. This ensures that the constraints will only accept valuations that are produced by the witness generator, which in turn captures the desired behavior of a ZK circuit program.

Definition 3 (Consistency). Let $P = (\Gamma, \mathbb{F}_p, \Phi, \omega)$ be a ZK circuit program. P is correct iff for all $\sigma \in \mathbb{F}_p^{|\Gamma|}$, we have $\sigma \vdash \Phi \sim \omega$.

The definition of consistency expresses the desired notion of correctness as it states that the constraint Φ agrees with the witness computation function ω for all possible signal valuations. This definition is similar to the definition of *strong safety* from prior work [16]. In particular, strong safety states that for every input to the circuit, (a) there is exactly one solution to the constraints and (b) that solution is equal to the witness produced by the witness generator on that input. However, we note that *strong safety* is too strong a requirement in practice: for many real world circuits, there can be multiple witnesses that satisfy the constraints which disagree on their assignment to the *intermediate signals* but agree on their assignments to the output signals. In contrast, our notion of consistency allows the two witnesses to disagree on the assignment to intermediate signals as long as both witnesses satisfy the constraints.

Skeleton $\mathcal{P} ::= D; S$
 Var Decl $D ::= \text{var } v[E]; \mid D_1; D_2$
 Statement $S ::= \Box_i \mid v \leftarrow E \mid S_1; S_2 \mid \text{if}(C) S_1 \text{ else } S_2 \mid \text{while}(C) S$
 Expression $E ::= E_1 \oplus E_2 \mid s \mid v \mid \alpha \mid E_1[E_2] \mid c$
 Conditional $C ::= E_1 \otimes E_2 \mid \neg C$

Fig. 8: Template syntax. c, s, v, α denote constants, signals, variables, and parameters respectively where $\oplus \in \{+, -, *, /, \backslash, \&, |\}$ and $\otimes \in \{\leq, <, >, \geq, \neq, =\}$. Holes are instantiated with terms from Fig. 9 and 10.

Constraint $\phi ::= E'_1 == E'_2$
 Expression $E' ::= s \mid v \mid \alpha \mid c \mid E'_1[E'_2]$
 | $E'_1 + E'_2 \mid E'_1 * E'_2$

Fig. 9: Constraint DSL

Assignments $A ::= L \leftarrow E \mid A; A$
 LHS Expression $L ::= s \mid L[E]$

Fig. 10: Witness DSL

ZK circuit templates. While ZK circuit programs cannot contain unbounded constraints or computation, many DSLs *do* allow developers to implement *templates* containing loops as long as these templates can be instantiated to ZK circuit programs at *compile time*. However, in general, it is often desirable to reason about the correctness of the template itself rather than its specific instantiations. For example, many libraries provide circuit templates that are instantiated by clients of that library, so reasoning about the correctness of a library in isolation requires the ability to reason about template correctness. Thus, we also generalize our formalization to circuit templates as follows.

Definition 4 (Template). A template $\mathcal{T}$ is a tuple $(V, \Gamma, \mathbb{F}_p, \mathcal{P}, \Sigma_\Phi, \Sigma_\omega)$ where:

- V is a set of template parameters;
- Γ is a set of signal declarations $s : \tau$, and $\mathbb{F}_p$ is the prime field for the circuit;
- $\mathcal{P}$ is a program with holes belonging to the syntax of Figure 8;
- Σ_Φ is a mapping from holes in $\mathcal{P}$ to constraints, as defined in Figure 9;
- Σ_ω is a mapping from holes in $\mathcal{P}$ to statements, as defined in Figure 10

We refer to $\mathcal{P}[\Sigma_\Phi]$ as a constraint template and $\mathcal{P}[\Sigma_\omega]$ as a witness template.

Template parameters $\alpha \in V$ are instantiated at compile-time; thus, they can never be modified. Signals can have type $\text{Array}(\tau)$, and we refer to a type τ as an *input type*, denoted $\text{lsInput}(\tau)$, if $\tau = \text{ln}$ or $\tau = \text{Array}(\tau')$ and $\text{lsInput}(\tau')$.

In most DSLs, the constraint and witness templates are required to have the same control flow structure; thus, we represent them using a shared *control-flow skeleton* $\mathcal{P}$. As shown in Figure 8, the skeleton involves loops, conditionals, constants, and importantly, *mutable* data variables denoted v . Typically, data variables are used as loop counters and for configuring different parts of the circuit. The constraint and witness templates are obtained by filling the holes in the shared control-flow skeleton with constraints (Fig. 9) or statements that manipulate signals (Fig. 10) respectively. Statements in the witness DSL can only perform assignments to signals, but not to data variables, which can only be modified in the shared control-flow skeleton.

```

1: procedure VERIFYCIRCUITTEMPLATE( $\mathcal{T}$ )
   input: A ZK circuit template  $\mathcal{T} = (V, \Gamma, \mathbb{F}_p, \mathcal{P}, \Sigma_\Phi, \Sigma_\omega)$ 
2:   # Step 1: Construct product of circuit and witness programs
3:    $S \leftarrow \{s \mid (s : \tau) \in \Gamma, \neg \text{IsInput}(\tau)\}$ 
4:    $\mathcal{P}_\otimes \leftarrow \mathcal{P}[\{(\square_i \mapsto A_i[S_w/S]; \phi_i[S_c/S]) \mid \Sigma_\Phi[\square_i] = \phi_i \wedge \Sigma_\omega[\square_i] = A_i\}]$ 
5:   # Step 2: Perform static analysis to simplify  $\mathcal{P}_\otimes$  into  $\mathcal{P}_\star$ 
6:    $\mathcal{E} \leftarrow \text{INFEREQUIVALENCES}(\mathcal{P}_\otimes, S)$ 
7:    $\mathcal{P}_\star \leftarrow \text{SIMPLIFY}(\mathcal{P}_\otimes, \mathcal{E})$ 
8:   # Step 3: Verify product program
9:    $\mathcal{P}_I \leftarrow \text{INSTRUMENT}(\mathcal{P}_\star)$ 
10:  return VERIFYPRODUCT( $\mathcal{P}_I$ )

```

Fig. 11: Top-level algorithm. Given signals S , we write S_x to denote $\{v_x \mid v \in S\}$

Definition 5 (Instantiation). A template instantiation is a pair $(\mathcal{T}, \nu)$ where $\mathcal{T}$ is a template, and ν maps template parameters V to values in $\mathbb{F}_p$, and $(\mathcal{T}, \nu) \hookrightarrow P$ indicates that P is the ZK circuit program obtained by evaluating $\mathcal{T}$ under ν .

Definition 6 (Correctness of template). A ZK circuit template $\mathcal{T}$ is correct if, for every ν such that $(\mathcal{T}, \nu) \hookrightarrow P$, P is correct according to Definition 3.

5 Verification Algorithm

Figure 11 shows our top-level algorithm for verifying the correctness of a circuit template. The algorithm starts by constructing a *product program* [19] that encodes the simultaneous execution of both the witness template $\mathcal{P}[\Sigma_\omega]$ and the constraint template $\mathcal{P}[\Sigma_\Phi]$ (lines 3–4). Because circuit languages force programmers to write constraint and witness templates with the same shared control-flow structure, constructing a suitable product program is very easy: We introduce two different copies S_c, S_w of the *non-input* signals and replace each hole $\square_i$ in $\mathcal{P}$ with $A_i[S_w/S]; \phi_i[S_c/S]$ where $A_i = \Sigma_\omega[\square_i]$ and $\phi_i = \Sigma_\Phi[\square_i]$.

The second step of the algorithm statically analyzes the constructed product program $\mathcal{P}_\otimes$ and tries to infer invariants of the form $s_c = s_w$ establishing that two copies of the same signal s have the same value. If so, we can treat signal s as an input signal after deleting all assignments to s_w and renaming the different copies s_c and s_w in $\mathcal{P}_\otimes$ both as s (performed by SIMPLIFY at line 7). Finally, the algorithm instruments the resulting program to contain suitable assertions and assumptions by calling INSTRUMENT at line 9 and attempts to discharge the assertions by calling VERIFYPRODUCT at line 10.

Assumptions. The rest of this section assumes that all signals and data variables are modeled as (possibly flattened) single-dimensional arrays. Following existing circuit languages, we also disallow multiple assignments to signals.

5.1 Inference of Equivalent Signals via Static Analysis

In this section, we present the INFEREQUIVALENCES algorithm from Figure 12, which uses lightweight static analysis to infer equivalent signal pairs $(s_c, s_w) \in (S_w \times S_c)$. The algorithm first constructs a so-called *symbolic store* Σ to represent values of array elements as symbolic expressions over the inputs

```

1: procedure INFEREQUIVALENCES( $P_{\otimes}, S$ )
2:    $\Sigma \leftarrow \text{CONSTRUCTSYMBOLICSTORE}(P_{\otimes}); \quad \mathcal{E} \leftarrow \emptyset$ 
3:   for  $s \in S$  do
4:     if ISEQUIVALENT( $\Sigma, s_w, s_c$ ) then  $\mathcal{E} \leftarrow \mathcal{E} \cup \{s\}$ 
5:   return  $\mathcal{E}$ 

```

Fig. 12: Algorithm for inferring equivalent signals

$$\begin{array}{c}
\frac{\varsigma \text{ fresh variable}}{\Sigma \vdash \star : \varsigma} (\text{UNKNOWN}) \quad \frac{\Sigma \vdash E : \varphi \quad (v, \varphi) \in \text{dom}(\Sigma)}{\Sigma \vdash v[E] : \Sigma(v, \varphi)} (\text{ARR-1}) \\
\\
\frac{\Sigma \vdash E : \varphi \quad (v, \varphi) \notin \text{dom}(\Sigma) \quad \neg \text{IsInput}(v) \quad \Sigma \vdash \star : \varphi'}{\Sigma \vdash v[E] : \varphi'} (\text{ARR-2}) \quad \frac{\Sigma \vdash E : \varphi \quad (v, \varphi) \notin \text{dom}(\Sigma) \quad \text{IsInput}(v)}{\Sigma \vdash v[E] : v[\varphi]} (\text{ARR-3}) \\
\\
\frac{\text{IsConst}(E) \vee \text{IsTemplParam}(E)}{\Sigma \vdash E : E} (\text{BASIC}) \quad \frac{\Sigma \vdash E_i : \varphi_i}{\Sigma \vdash f(E_1, \dots, E_n) : f(\varphi_1, \dots, \varphi_n)} (\text{ARITH})
\end{array}$$

Fig. 13: Look-up rules. Every application of UNKNOWN produces a fresh variable.

(line 2) and then checks pair-wise equality between them (line 4). The symbolic expressions φ in our analysis are defined by the grammar:

$$\varphi := \alpha \mid c \mid \varsigma \mid s_{in}[\varphi] \mid \varphi \oplus \varphi$$

Here, s_{in} is an input signal, α a template parameter, c a constant, ς an unknown, and $\oplus$ an arithmetic operator. Symbolic expressions represent immutable values, ensuring that syntactically identical expressions are equal. To improve precision and enable relational reasoning, fresh variables ς are used to represent unknowns instead of the standard top element $\top$ in abstract interpretation. For a symbolic expression φ , $\text{Unknown}(\varphi)$ indicates that it corresponds to a fresh variable ς .

The symbolic store Σ maps array elements (v, φ) to symbolic expressions, where v is a signal or data variable, and φ represents an array index. For instance, $(s_w, 0) \mapsto s_{in}[1]$ indicates that index 0 of s_w is (transitively) assigned to the second element of s_{in} . Figure 14 defines a flow-sensitive analysis for constructing Σ , using lookup rules from Figure 13 to perform on-demand initialization of input arrays, as their sizes are not statically known. In Figure 14, the rule ASGN-SIG processes assignments to *signals*, updating the entry (s_w, φ_1) with φ_2 via a *strong update* [24], since circuit languages ensure signals are assigned only once. For data variables, the ASGN-VAR rule adopts a conservative approach, handling potential aliasing by updating (v, φ) and invalidating other entries (v, φ_i) that might alias φ . The CSTR rule tracks equality constraints on signals s_c . The final three rules handle sequencing, conditionals, and loops, where the join operator is defined in Figure 15. Finally, the WHILE rule requires the symbolic store Σ_1 to be a fixed-point solution for the loop.

Once the symbolic store is constructed, the ISEQUIVALENT procedure (summarized in Figure 16) uses these symbolic expressions to deduce that two signals s_c, s_w are equivalent, denoted $\Sigma \vdash s_c \sim s_w$. According to Figure 16, signals s_c, s_w are equivalent if (a) they have the same set of written indices and (b) their values

$$\begin{array}{c}
\frac{\Sigma \vdash E_1 : \varphi_1 \quad \Sigma \vdash E_2 : \varphi_2}{\Sigma \vdash s_w[E_1] \leftarrow E_2 : \Sigma[(s_w, \varphi_1) \leftarrow \varphi_2]} (\text{ASGN-SIG}) \\
\\
\frac{\Sigma \vdash \star : \varphi'}{\Sigma \vdash \text{Havoc}(v, \varphi) : \Sigma[(v, \varphi) \leftarrow \varphi']} (\text{HAVOC}) \\
\\
\frac{\begin{array}{c} \Sigma \vdash E_1 : \varphi \quad \Sigma \vdash E_2 : \varphi' \\ \Theta = \{(\varphi_i \mid (v, \varphi) \in \text{Dom}(\Sigma) \wedge \text{SAT}(\varphi = \varphi_i))\} \\ \Sigma \vdash \text{Havoc}(v, \varphi_1); \dots; \text{Havoc}(v, \varphi_{|\Theta|}) : \Sigma' \end{array}}{\Sigma \vdash v[E_1] \leftarrow E_2 : \Sigma'[(v, \varphi) \leftarrow \varphi']} (\text{ASGN-VAR}) \\
\\
\frac{\Sigma \vdash E : \varphi \quad \Sigma \vdash E' : \varphi' \quad \Sigma' = \Sigma[(s_c, \varphi) \leftarrow \varphi']}{\Sigma \vdash s_c[E] == E' : \Sigma'} (\text{CSTR}) \\
\\
\frac{\Sigma \vdash S_1 : \Sigma_1 \quad \Sigma_1 \vdash S_2 : \Sigma_2}{\Sigma \vdash S_1; S_2 : \Sigma_2} (\text{SEQ}) \quad \frac{\Sigma \vdash S_1 : \Sigma_1 \quad \Sigma \vdash S_2 : \Sigma_2 \quad \Sigma' \leftarrow \Sigma_1 \sqcup \Sigma_2}{\Sigma \vdash \text{if}(C) S_1 \text{ else } S_2 : \Sigma'} (\text{IF}) \quad \frac{\begin{array}{c} \Sigma \sqsubseteq \Sigma_1 \\ \Sigma_1 \vdash S : \Sigma_2 \\ \Sigma_2 \sqsubseteq \Sigma_1 \end{array}}{\Sigma \vdash \text{while}(C) S : \Sigma_1} (\text{WHILE})
\end{array}$$

Fig. 14: Symbolic store construction. $\Sigma_1 \sqsubseteq \Sigma_2$ holds if, for all elements $(v, \varphi) \in \text{Dom}(\Sigma_1)$, $\Sigma_1(v, \varphi) \sqsubseteq \Sigma_2(v, \varphi)$. $\varphi_1 \sqsubseteq \varphi_2$ holds if $\varphi_1 = \varphi_2$ or $\text{Unknown}(\varphi_2)$.

$$\begin{array}{c}
\frac{\varphi_1 = \varphi_2}{\varphi_1 \sqcup \varphi_2 = \varphi_1} (\text{SYMEXP-1}) \quad \frac{\varphi_1 \neq \varphi_2 \quad \Sigma \vdash \star : \varphi}{\varphi_1 \sqcup \varphi_2 : \varphi} (\text{SYMEXP-2}) \\
\\
\frac{(v, \varphi) \notin \text{Dom}(\Sigma_i) \quad \Sigma_i \vdash \star : \varphi'}{(\Sigma_1 \sqcup \Sigma_2)[(v, \varphi)] = \varphi'} (\text{ONE}) \quad \frac{(v, \varphi) \in \text{Dom}(\Sigma) \cap \text{Dom}(\Sigma')}{(\Sigma \sqcup \Sigma')[(v, \varphi)] = \Sigma[(v, \varphi)] \sqcup \Sigma'[(v, \varphi)]} (\text{BOTH})
\end{array}$$

Fig. 15: Rules defining join operation on symbolic stores

at these indices are equal. Here, we use the notation $\varphi \equiv \varphi'$ to indicate that φ and φ' are equivalent expressions (e.g., $\alpha + 1 \equiv 1 + \alpha$). The theorems stating the soundness of our analysis can be found in the appendix, along with their proofs.

5.2 Product Program Simplification and Instrumentation

The next step in the algorithm uses the static analysis results to simplify the product program. Figure 17 presents the SIMPLIFY procedure as inference rules $\mathcal{E} \vdash S \rightsquigarrow S'$, where S simplifies to S' under inferred equivalences $\mathcal{E}$. The procedure deletes assignments like $s_w[E] \leftarrow E$ if s_w and s_c are equivalent (S-ASG1) and substitutes all occurrences of s_c and s_w with s for any $s \in \mathcal{E}$ (S-EXPR). Next, given the simplified program $\mathcal{P}_\star$, the INSTRUMENT procedure generates the instrumented program $\mathcal{P}_I$ such that $\mathcal{P}_I$ is safe if and only if the original template is correct. This procedure is summarized in Figure 18 and appends an assertion at the end of the product program stating pair-wise equivalence between the output signals. It also recursively instruments the body of $\mathcal{P}_\star$, as summarized in the first three rows of Figure 18. In particular, constraints of the form $e_1[S_c] == e_2[S'_c]$ are rewritten into an assertion $\text{assert}(e_1[S_w] = e_2[S'_w])$ on the witness variables and an assumption $\text{assume}(e_1[S_c] = e_2[S'_c])$ on the constraint variables. The assumption ensures counterexamples are limited to executions where $\llbracket \Phi \rrbracket(\sigma)$ holds,

$$\frac{\begin{array}{l} \forall (s_w, \varphi) \in \text{dom}(\Sigma). \exists (s_c, \varphi') \in \text{dom}(\Sigma). \quad \varphi \equiv \varphi' \wedge \Sigma(s_c, \varphi) \equiv \Sigma(s_w, \varphi) \\ \forall (s_c, \varphi) \in \text{dom}(\Sigma). \exists (s_w, \varphi') \in \text{dom}(\Sigma). \quad \varphi \equiv \varphi' \wedge \Sigma(s_w, \varphi) \equiv \Sigma(s_c, \varphi) \end{array}}{\Sigma \vdash s_c \sim s_w} \text{(SIG-EQUIV)}$$

Fig. 16: Rule describing the ISEQUIVALENT procedure

$$\frac{S = \{s \mid s \in \text{Sigs}(E) \wedge s \in \mathcal{E}\}}{\mathcal{E} \vdash E \rightsquigarrow E[S/S_c, S/S_w]} \text{(S-EXPR)} \quad \frac{s \in \mathcal{E}}{\mathcal{E} \vdash s[E_1] \leftarrow E_2 \rightsquigarrow \text{skip}} \text{(S-ASG1)}$$

$$\frac{s \notin \mathcal{E} \quad \mathcal{E} \vdash E_1 \rightsquigarrow E'_1 \quad \mathcal{E} \vdash E_2 \rightsquigarrow E'_2}{\mathcal{E} \vdash s[E_1] \leftarrow E_2 \rightsquigarrow s[E'_1] \leftarrow s[E'_2]} \text{(S-ASG2)}$$

Fig. 17: SIMPLIFY procedure. $\text{Sigs}(E)$ returns the normalized (i.e., without c, w subscripts) versions of the signals in E . We omit the trivial rules (e.g., for sequencing).

enforcing the antecedent in Def. 2. On the other hand, the assertion enforces that witnesses satisfy constraints $\llbracket \Phi \rrbracket(\sigma')$, corresponding to the first conjunct in the consequent Def. 2. Finally, the second conjunct in the consequent of Def. 2 is enforced by the assertions placed at the end of the product program.

5.3 Verification of Instrumented Product Program

The verification algorithm (Figure 19) operates in two phases. In the first phase, it populates an environment Λ with information about loop counters, calling `InferBounds` (line 7) to compute lower and upper bounds for each counter κ via interval analysis [25] and invoking `InferStepSize` to determine κ 's step size. The second phase (lines 9–15) computes inductive loop invariants, initializing them in the first iteration and weakening each invariant until soundness is ensured. Specifically, for each loop L , `INFERLOOPINV` is invoked at line 13 to compute an inductive invariant for L *assuming* that $\Psi[L]$ over-approximates L 's true precondition. These loop pre-conditions (Ψ) are computed through standard post-condition computation, using invariants $\mathcal{I}$ for preceding loops. However, since the “invariants” computed in the first iteration may be stronger than the true invariant, these pre-conditions may also be too strong. Hence, the algorithm weakens both the pre-conditions and the invariants until a fixed-point is reached. Upon convergence, it generates verification conditions and checks their validity.

Inferring loop invariants. Figure 20 outlines our procedure for generating loop invariants, based on a standard Houdini-style approach [26] with two distinguishing features. First, it uses a novel technique (`CONJECTUREPREDICATES`) to construct the predicate universe; second, it includes a boolean parameter `check-pre` to control whether predicates are filtered using pre-condition ψ . During the first invocation, `check-pre` is **false**, so the inferred invariant could be too

$$\begin{array}{ll} \text{Instr}(e_1[S_c] == e_2[S'_c]) & \triangleq \text{assume}(e_1[S_c] = e_2[S'_c]) \text{assert}(e_1[S_w] = e_2[S'_w]); \\ \text{Instr}(s[E_1] \leftarrow E_2) & \triangleq s[E_1] \leftarrow E_2 \\ \text{Instr}(S_1; S_2) & \triangleq \text{Instr}(S_1); \text{Instr}(S_2) \\ \text{INSTRUMENT}(P_\otimes) & \triangleq \text{Instr}(P_\otimes); \text{assert}(\forall s \in \text{OutputSignals}(P_\otimes). s_w \equiv s_c) \end{array}$$

Fig. 18: Instrumentation procedure, INSTRUMENT

```

1: procedure VERIFYPRODUCT( $P_{\otimes}$ )
2:   #  $\mathcal{I}, \Psi$  map each loop to their invariant and pre-conditions respectively
3:    $(\mathcal{I}, \Psi) \leftarrow (\emptyset, \emptyset); \Lambda \leftarrow \emptyset$ 
4:   # Phase 1: Infer information about loop counters
5:   for  $L \in \text{Loops}(P_{\otimes})$  do
6:      $(\kappa, \mathcal{I}[L]) \leftarrow (\text{InferLoopCounter}(L), \text{true})$ 
7:      $\Lambda[\kappa] \leftarrow (\text{InferBounds}(\kappa, L), \text{InferStepSize}(\kappa, L))$ 
8:   # Phase 2: Weaken invariants until they are sound
9:    $\text{done} \leftarrow \text{false}; \text{first} \leftarrow \text{true}$ 
10:  while  $\neg \text{done}$  do
11:     $(\Psi, \text{done}) \leftarrow (\text{ComputeSP}(P_{\otimes}, \mathcal{I}), \text{true})$ 
12:    for  $L \in \text{Loops}(P_{\otimes})$  do
13:       $I \leftarrow \text{INFERLOOPINV}(L, \Psi[L], \Lambda, \neg \text{first})$ 
14:      if  $I \neq \mathcal{I}[L]$  then  $\text{done} \leftarrow \text{false}; \mathcal{I}[L] \leftarrow I$ 
15:     $\text{first} \leftarrow \text{false}$ 
16:  # Generate VCs and check their validity
17:  return  $\text{Valid}(\text{GenVC}(P_{\otimes}, \mathcal{I}))$ 

```

Fig. 19: Verification algorithm; procedures in sans serif font are explained in text.

```

1: procedure INFERLOOPINV( $L, \psi, \Lambda, \text{check-pre}$ )
2:   input: Loop  $L$  with condition  $C$  and body  $B$ , pre-condition  $\psi$ ,
3:   counter environment  $\Lambda$ , boolean  $\text{check-pre}$ 
4:   # Conjecture predicates that are likely to be part of the invariant
5:    $\Pi \leftarrow \text{CONJECTUREPREDICATES}(L, \Lambda)$ 
6:   # Remove predicates not implied by pre-condition
7:   if  $\text{check-pre}$  then  $\Pi \leftarrow \{\varphi \mid \varphi \in \Pi \wedge \psi \Rightarrow \varphi\}$ 
8:   # Houdini-style fixed-point computation; done initialized to false
9:   while  $\neg \text{done}$  do
10:     $\text{done} \leftarrow \text{true}$ 
11:    for  $\varphi \in \Pi$  do
12:      if  $\not\models \{C \wedge \bigwedge_{p_i \in \Pi} p_i\} B \{\varphi\}$  then  $\Pi \leftarrow \Pi \setminus \{\varphi\}; \text{done} \leftarrow \text{false}$ 
13:  return  $\bigwedge_{p \in \Pi} p$ 

```

Fig. 20: Invariant inference algorithm. If Π is empty, then we define $\bigwedge_{p \in \Pi} p$ to be **True**.

strong (satisfying consecution but not initiation). In subsequent calls, **check-pre** is **true**, ensuring that predicates failing initiation are eventually filtered out.

CONJECTUREPREDICATES (Figure 21) generates candidate predicates for a loop L , returning a set of *guarded array equality* (GAE) predicates which are of the form $\forall \bar{x}. \phi(\bar{x}) \rightarrow s_c[f(\bar{x})] = s_w[f(\bar{x})]$. To explain our algorithm, we utilize the example in Figure 22 for which the required invariant is:

$$\forall x, y. (0 \leq x < i \leq N \wedge 0 \leq y < M) \vee (x = i \wedge 0 \leq y < j \leq M) \rightarrow s_w[N * x + y] = s_c[N * x + y]$$

Here, the guard captures all loop counter values c_x, c_y for which $s_w[N * c_x + c_y]$ was written before the current iteration. To infer such predicates, the algorithm first identifies all writes in the loop (line 2),

```

for (var  $i = 0; i < N; i++$ ) {
  for (var  $j = 0; j < M; j++$ ) {
     $s_w[N * i + j] \leftarrow a;$ 
     $s_c[N * i + j] == a;$ 
  }
}

```

Fig. 22: Loop Example

```

1: procedure CONJECTUREPREDICATES( $L, \Lambda$ )
   input: Loop  $L$  and counter environment  $\Lambda$ 
   output: Candidate guarded array equality predicates  $\Pi$ 
2:    $\mathcal{W} \leftarrow \text{SignalWrites}(L)$     $\Pi \leftarrow \{\}$ ;
3:   for  $(s_w, f(\kappa_1, \dots, \kappa_n)) \in \mathcal{W}$  do
4:     # Infer constraints over counters  $\kappa_1, \dots, \kappa_n$ , sorted outermost to innermost
5:      $(\varphi, \varphi_a) \leftarrow (\text{false}, \text{true})$ 
6:     for  $\kappa_i \in [\kappa_1, \dots, \kappa_n]$  do
7:       # generate constraints  $\varphi_{\kappa_i}$  on  $\kappa_i$  assuming  $\varphi_a = \bigwedge_{i=1}^n x_i = \kappa_i$ 
8:        $(l_i, u_i, s_i) \leftarrow \Lambda[\kappa_i]$ 
9:        $\varphi_{\kappa_i} \leftarrow \varphi_a \wedge (l_i \leq x_i < \kappa \leq u_i \wedge x_i - l_i \bmod s_i = 0)$ 
10:      for  $\kappa_j \in [\kappa_{i+1}, \dots, \kappa_n]$  do
11:         $(l_j, u_j, s_j) \leftarrow \Lambda[\kappa_j]$ 
12:         $\varphi_{\kappa_j} \leftarrow \varphi_{\kappa_i} \wedge (l_j \leq x_j < u_j \wedge x_j - l_j \bmod s_j = 0)$ 
13:      # Add constraints  $\varphi_{\kappa_j}$  to  $\varphi$  and add assumption  $x_i = \kappa_i$  to  $\varphi_a$ 
14:       $(\varphi, \varphi_a) \leftarrow (\varphi \vee \varphi_{\kappa_j}, \varphi_a \wedge x_i = \kappa_i)$ ;
15:    $\Pi \leftarrow \Pi \cup \{\forall x_1, \dots, x_n. \varphi \rightarrow s_w[f(x_1, \dots, x_n)] = s_c[f(x_1, \dots, x_n)]\}$ 
16: return  $\Pi$ 

```

Fig. 21: Algorithm for generating guarded array equality predicates.

represented as tuples $(s_w, f(\kappa_i, \dots, \kappa_j))$, where $f(\kappa_i, \dots, \kappa_j)$ is the symbolic expression for the index written in s_w . In this example, the only write is to s_w at $N * i + j$, so the algorithm conjectures $s_w[N * i + j] = s_c[N * i + j]$ as the array equality component of the GAE predicate.

For each inferred array equality predicate, the algorithm constructs its guard φ by iterating over loop counter variables from outermost to innermost. For each counter κ_i , it synthesizes a predicate φ_{κ_i} capturing all feasible values for counters $[\kappa_i, \dots, \kappa_n]$, assuming parent counters hold their current iteration values (φ_a). Lines 10–14 compute φ_{κ_i} using loop bounds and step sizes from Λ , ensuring the predicate accounts for all inner counter values during successful executions of prior iterations. The final guard φ is updated as $\varphi \vee \varphi_{\kappa_i}$, while φ_a adds the assumption $x_i = \kappa_i$. Going back to the example, the counter environment maps i to $[0, N)$ and step size 1, and j to $[0, M)$ and step size 1. The procedure first generates φ_i as $0 \leq x_i < i \leq N \wedge 0 \leq x_j < M$. Next, it updates φ_a with $x_i = i$ and computes φ_j for the inner loop as $x_i = i \wedge 0 \leq x_j < j \leq M$. Finally, φ becomes $\varphi_i \vee \varphi_j$, yielding the desired guard:

$$(0 \leq x_i < i \leq N \wedge 0 \leq x_j < M) \vee (x_i = i \wedge 0 \leq x_j < j \leq M)$$

We refer the interested reader to 10.6 in the appendix for an example demonstrating a more complex loop invariant.

6 Implementation

We implemented our approach in a tool called ZEQUAL and built it on top of the Circom compiler [27]. ZEQUAL discharges SMT queries via Z3 [28] in the theory of integers. Finite field operations are modeled by performing all operations modulo the BN254 prime. For expressions composed of additions, subtractions, or multiplications, the modulo reduction is moved to the outermost expression

to reduce the number of large mod operations. In cases where an operation is not supported by the theory of integers (e.g. bitwise arithmetic, exponentiation, finite field inverse), we model these operations with uninterpreted functions and include appropriate axioms.

Although Section 5 describes our procedure as operating on a single product program, real-world circuits and witness generators are often composed of sub-circuit invocations. ZEQUAL verifies such circuits by assuming the consistency of any invoked subcircuit. A circuit can therefore be verified by checking the consistency of all invoked sub-circuits in addition to the circuit itself.

7 Evaluation

In this section, we present the results of an experimental evaluation that is designed to answer the following research questions: (1) Can ZEQUAL be used to verify real-world circuits? (2) Does ZEQUAL uncover buggy ZK circuit templates in the wild? (3) How important is the proposed static analysis for successful verification? (4) Can static analysis be used *on its own* to prove consistency? To answer questions (1) and (2), we analyze what percentage of circuits ZEQUAL can verify/falsify. To answer the latter two questions, we perform two ablation studies that disable static analysis and deductive verification respectively.

Benchmarks. We evaluate ZEQUAL on Circom programs gathered from the 20 most popular Circom projects (measured by GitHub stars). After excluding four repositories that use unsupported features (e.g., incompatible with ZEQUAL’s version of the Circom compiler), we retain 464 templates from 16 unique repositories, spanning over 290K lines of code. Collectively, these projects span a variety of applications, including cryptographic primitives, games, and machine learning components. For all projects, ZEQUAL attempts to verify the consistency of each template declared within the source code. Since all templates rely on primitives provided by circomlib, which serves as a foundational library for Circom development, ZEQUAL employs stubs of circomlib circuits when analyzing other projects. These stubs capture the behavior of circomlib components, enabling ZEQUAL to reason about templates in other repositories without requiring their full implementation. To evaluate ZEQUAL on these benchmarks, we make *minor* semantics-preserving modifications to a small subset of the templates in order to ensure that they comply with ZEQUAL’s assumptions (e.g., our implementation assumes that array sizes only reference template parameters rather than local variables).

Experimental Setup. The experiments were conducted on a MacBook Pro® with an M1 Max CPU, 64 GB of memory, and MacOS 12.3.1. Each benchmark was allocated a runtime of 10 minutes. Each experiment requires verifying the correctness of a specific template, assuming all external templates are correct.

Main Results. Table 1 summarizes the results of our evaluation. Out of 464 benchmarks, ZEQUAL successfully verifies the correctness of 306 templates, meaning it can verify correctness for *any* instantiation of the templates in 66% of the benchmarks. ZEQUAL is also highly efficient, with an average verification time of

Project	# templates	LoC	# verified	# cex	# failed	Verif. time
tornado-core	5	102	5 (100%)	0 (0%)	0 (0%)	0.40
semaphore	2	50	2 (100%)	0 (0%)	0 (0%)	2.41
circomlib	103	28,660	69 (67%)	24 (23%)	10 (10%)	7.28
maci	52	2,171	40 (77%)	9 (17%)	3 (6%)	1.74
stealthdrop	36	161,251	17 (47%)	13 (36%)	6 (17%)	13.58
circom-ecdsa	63	68,359	36 (57%)	22 (35%)	5 (8%)	1.65
hydra-s1-zkps	4	152	4 (100%)	0 (0%)	0 (0%)	2.31
zkshield	5	238	2 (40%)	1 (20%)	2 (40%)	6.14
circomlib-ml	37	897	34 (92%)	3 (8%)	0 (0%)	5.75
eigen-zkvm	21	23,856	11 (52%)	6 (29%)	4 (19%)	10.85
zk-nullifier-sig	44	1,762	24 (55%)	13 (30%)	7 (16%)	16.49
tornado-nova	7	190	5 (71%)	2 (29%)	0 (0%)	1.77
darkforest-eth	22	699	13 (59%)	8 (36%)	1 (5%)	3.84
ed25519-circom	20	2,186	8 (40%)	10 (50%)	2 (10%)	3.86
unirep	14	595	12 (86%)	1 (7%)	1 (7%)	1.42
zk-hunt	29	995	24 (83%)	4 (14%)	1 (3%)	1.68
Overall	464	292,163	306 (66%)	116 (25%)	42 (9%)	5.71

Table 1: Results. The column called “#verified” shows the number of benchmarks for which ZEQUAL is able to prove correctness, “# cex” shows the number of benchmarks for which ZEQUAL is unable to prove the validity of the generated VC and “# failed” shows the number of benchmarks for which ZEQUAL times out or returns unknown. The last column called “verif. time” shows the average running time of ZEQUAL for those benchmarks that were successfully verified.

just 5.71 seconds per template. For the remaining 158 benchmarks, verification does not succeed. In 42 cases (9%), ZEQUAL exceeds the 10-minute time limit or the tool returns unknown, primarily due to the Z3 timing out on SMT queries. In the remaining 116 benchmarks (25%), ZEQUAL identifies counterexamples to the validity of the generated VC. These counterexamples occur for two reasons: (1) some template instantiations are genuinely incorrect, or (2) the template is correct, but verification fails due to analysis imprecision, such as weak loop invariants.

Failure analysis. Next, we examine the scenarios in which ZEQUAL is unable to prove correctness. The findings from this failure analysis are summarized in Figure 23. Among the cases where verification fails, 22% are true positives, which are discussed in more detail below. For the remaining benchmarks, ZEQUAL fails to verify correctness for four primary reasons:

Reason for failure	# (%)
True positive	26 (22%)
Insufficient loop invariant	61 (53%)
Imprecise loop bound	10 (9%)
Imprecise template stub	8 (7%)
Imprecise operation modeling	11 (9%)

Fig. 23: Results of the failure analysis.

- *Insufficient loop invariant:* For 61 of the failure cases (53% of false positives), the loop invariant inferred by ZEQUAL is too weak. For instance, several benchmarks require invariants that capture precise valuations of s_c and s_w

to establish the consistency of a signal s across *multiple* loops. We refer the interested reader to Section 10.4 for a representative example.

- *Imprecise loop bounds*: Recall that our approach relies on static analysis of loop counters to infer bounds and step sizes as a precursor to loop invariant generation. 10 of the failure cases (i.e., 9% of false positives) are due to imprecision in the static analysis of loop counters.
- *Imprecise template stubs*: Recall that our approach utilizes stubs of circomlib templates when analyzing other projects. In some cases, the stubs are not sufficiently precise, accounting for 7% of false positives.
- *Imprecise modeling*: As discussed in Section 6, ZEQUAL uses uninterpreted functions to model operations that are not supported by the theory of integers. Roughly 9% of false positives are caused by imprecision in the SMT encoding.

True positive analysis. Among the 116 potentially vulnerable circuits we manually inspected, 26 were confirmed to correspond to real bugs. However, since ZEQUAL assumes that external template invocations are safe, these 26 templates could also have cascading effects on other benchmarks. We therefore reran ZEQUAL without this assumption, and identified an additional 20 buggy benchmarks. Hence, ZEQUAL identified a total of 46 buggy templates in the wild.

Of the 46 falsified templates, 41 exhibit violations of the determinism property explored in prior work [8, 10] under *some* instantiation of the template parameters. On the other hand, 5 benchmarks are incorrect despite being deterministic and fail to properly validate their inputs. In practice, an attacker can easily take advantage of such bugs in the validation logic to trick the verifier into accepting proofs that should be classified as invalid. An example of an incorrect but deterministic benchmark can be found in Section 10.5 in the appendix.

Comparison with prior work. As discussed previously, 41 out of the 46 incorrect templates also violate the determinism property studied in prior work [8, 10]. Therefore, *in principle*, these 41 bugs could be uncovered by an existing tool like QED² and CIVER, which specialize in refuting determinism. However, existing tools for determinism checking are limited to analyzing fully instantiated circuits and cannot directly handle *templates*.

To evaluate how ZEQUAL compares with existing determinism checkers, we instantiated the 41 non-deterministic templates with the *smallest* parameter values that expose the bug. This approach avoids scalability issues caused by larger parameter values while ensuring that the bug remains detectable. Using state-of-the-art tools, QED² and CIVER, we then attempted to identify the non-determinism bugs within a time limit of 10 minutes. Among the 41 non-deterministic circuits, QED² successfully detected the issue in 22 (54%) circuits but failed to do so for the remaining 19 (46%) cases. Similarly, CIVER successfully detected the issue in 27 (66%) circuits but failed to do so for the remaining 14 (34%) cases.

These results highlight a significant limitation of existing open source determinism checking tools: their ability to refute determinism diminishes in practice

due to scalability challenges, even when working with specific circuit instantiations. In contrast, ZEQUAL directly analyzes templates and verifies a much stronger property for *any* instantiation, without requiring manually-crafted parameter templates. However, ZEQUAL cannot automatically refute correctness, even when restricted to determinism, making it a complementary approach rather than a replacement.

Ablation Study. To assess the significance of the key design choices in ZEQUAL, we compare its performance against two ablated versions:

- ZEQUAL-NSA: This variant disables the static analysis described in Section 5.1. Without static analysis, ZEQUAL-NSA bypasses simplifications of the product program and directly invokes the verification procedure. This evaluation highlights the contribution of static analysis in reducing verification complexity.
- ZEQUAL-OSA: This variant relies solely on the static analysis results to determine correctness. Specifically, it uses the ISEQUIVALENT procedure to verify whether *all* signals in the ZK circuit are equivalent. If this equivalence holds, all instrumented assertions reduce to *true*, and the circuit is deemed correct. This approach examines the standalone utility of static analysis without leveraging the full verification pipeline in ZEQUAL.

The results of the ablation study, shown in Figure 24, highlight the importance of both the static analysis and the proposed verification pipeline. Specifically, ZEQUAL-NSA verifies over 100 fewer benchmarks than ZEQUAL due to the increased complexity of the product program, which leads to a significantly higher number of timeouts. On the other hand, ZEQUAL-OSA is much faster since it relies solely on lightweight static analysis, but it verifies only half as many benchmarks as ZEQUAL. These results demonstrate that static analysis alone is insufficient for proving correctness, reinforcing the necessity of the full verification pipeline.

Tool	# verified
ZEQUAL-NSA	205 (44%)
ZEQUAL-OSA	150 (32%)
ZEQUAL	306 (66%)

Fig. 24: Ablation study

8 Related Work

Formal Verification of ZK Circuits. Recent work has explored using formal verification to check the correctness of ZK circuits [8, 10–12, 29–32]. One line of work uses interactive theorem provers to verify functional correctness [12, 29, 30]. For example, CODA [12] uses a custom circuit DSL embedded in Coq which is equipped with a rich type system for specifying circuit properties. Its type checker uses a combination of tactics along with manually proved lemmas to check correctness in Coq. Another line of work uses SMT solvers for automated verification [8, 10, 11, 31, 32], focusing on specific circuit properties rather than full functional correctness. The most related work to ours is QED² [8], which checks the determinism of ZK circuits using a combination of static analysis and SMT solvers like ZEQUAL. However, unlike ZEQUAL, QED² checks a weaker property and is limited to instantiated circuits (without loops), whereas ZEQUAL can directly analyze circuit templates.

Relational Verification. Checking relational properties between programs often involves building product programs [33–37], and a key challenge is determining a suitable alignment between programs. For instance, Churchill et al. [34] check program equivalence by using manually provided tests to generate program traces and synthesize alignment predicates from the trace states. They then lift the alignment predicates back to the source code to construct the product program. In contrast, ZEQUAL leverages the natural alignment between constraints and witness generators in circuit DSLs and performs static analysis to substantially simplify the product program.

Array Invariants. There is an extensive body of work on generating quantified invariants for loops with array variables [38–43]. For example, Flanagan et al. [39] use predicate abstraction and Skolemization to construct quantified invariants, while Gulwani et al. [41] employ a quantified abstract domain to express universally quantified formulas over arrays. Although these techniques could be applied in our setting, we leverage a domain-specific insight: Most loop invariants express equality between the witness and constraint components of signal arrays. This allows us to generate high-quality templates and efficiently construct loop invariants in most cases.

9 Conclusion

This paper presents ZEQUAL, a framework for verifying the consistency of zero-knowledge circuit templates by combining static analysis and deductive verification. ZEQUAL formalizes the consistency requirement for circuit DSLs and introduces a verification method that directly operates on templates, providing guarantees for any circuit instantiated from them. We have evaluated the proposed approach through an extensive experimental study on 464 real-world benchmarks drawn from popular Circom projects. The results show that ZEQUAL terminates on 91% of the benchmarks within a 10-minute time limit and successfully verifies 72% of the cases where it terminates. For benchmarks where verification fails, a systematic failure analysis reveals that roughly 22% of counterexamples correspond to true positives. ZEQUAL also uncovers several bugs that a state-of-the-art tool, QED², cannot detect. This capability arises from two key advantages: First, ZEQUAL is not limited to verifying determinism, and, second, it can reason about all possible instantiations of the template in a scalable manner. Additionally, ablation studies validate the design choices behind ZEQUAL, showing that the combination of lightweight static analysis and deductive verification is critical for making the approach effective.

Acknowledgments. This work was conducted in a research group supported by NSF awards CCF-1762299, CCF-1918889, CNS-1908304, CCF-1901376, CNS-2120696, CCF- 2210831, and CCF-2319471.

Disclosure of Interests. All authors are employed by Veridise, a company that provides formal verification services. The authors have no other competing interests to declare that are relevant to the content of this article.

Bibliography

- [1] J. Groth, “Short non-interactive zero-knowledge proofs,” in *Advances in Cryptology - ASIACRYPT 2010* (M. Abe, ed.), (Berlin, Heidelberg), pp. 341–358, Springer Berlin Heidelberg, 2010.
- [2] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, “Scalable, transparent, and post-quantum secure computational integrity.” Cryptology ePrint Archive, Paper 2018/046, 2018.
- [3] K. W. Jie, “Openclimate-sg/datawhistleblowing: Work done for the yale open climate collabathon.”
- [4] T. Datta, B. Chen, and D. Boneh, “VerITAS: Verifying image transformations at scale.” Cryptology ePrint Archive, Paper 2024/1066, 2024.
- [5] E. Ben Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza, “Zerocash: Decentralized anonymous payments from bitcoin,” in *2014 IEEE Symposium on Security and Privacy*, pp. 459–474, 2014.
- [6] C. Smith, “Zero-knowledge rollups.”
- [7] S. Chaliasos, J. Ernstberger, D. Theodore, D. Wong, M. Jahanara, and B. Livshits, “Sok: What don’t we know? understanding security vulnerabilities in snarks,” 2024.
- [8] S. Pailoor, Y. Chen, F. Wang, C. Rodríguez, J. V. Gaffen, J. Morton, M. Chu, B. Gu, Y. Feng, and I. Dillig, “Automated detection of under-constrained circuits for zero-knowledge proofs.” Cryptology ePrint Archive, Paper 2023/512, 2023.
- [9] H. Wen, J. Stephens, Y. Chen, K. Ferles, S. Pailoor, K. Charbonnet, I. Dillig, and Y. Feng, “Practical security analysis of zero-knowledge proof circuits.” Cryptology ePrint Archive, Paper 2023/190, 2023.
- [10] M. Isabel, C. Rodriguez-Nunez, and A. Rubio, “ Scalable Verification of Zero-Knowledge Protocols ,” in *2024 IEEE Symposium on Security and Privacy (SP)*, (Los Alamitos, CA, USA), pp. 1794–1812, IEEE Computer Society, May 2024.
- [11] F. H. Soureshjani, M. Hall-Andersen, M. Jahanara, J. Kam, J. Gorzny, and M. Ahmadvand, “Automated analysis of halo2 circuits.” Cryptology ePrint Archive, Paper 2023/1051, 2023.
- [12] J. Liu, I. Kretz, H. Liu, B. Tan, J. Wang, Y. Sun, L. Pearson, A. Miltner, I. Dillig, and Y. Feng, “Certifying zero-knowledge circuits with refinement types,” in *2024 IEEE Symposium on Security and Privacy (SP)*, pp. 1741–1759, 2024.
- [13] B. Parno, C. Gentry, J. Howell, and M. Raykova, “Pinocchio: Nearly practical verifiable computation.” Cryptology ePrint Archive, Paper 2013/279, 2013. <https://eprint.iacr.org/2013/279>.
- [14] V. Buterin, “Quadratic arithmetic programs: From zero to hero.” <https://medium.com/@VitalikButerin/quadratic-arithmetic-programs-from-zero-to-hero-f6d558cea649>, Dec 2016.

- [15] ConsenSys, *gnark: A Framework for Zero-Knowledge Proofs*, 2024. Documentation for circuit concepts in gnark.
- [16] M. Bellés-Muñoz, M. Isabel, J. L. Muñoz-Tapia, A. Rubio, and J. Baylina, “Circom: A circuit description language for building zero-knowledge applications,” *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 6, pp. 4733–4751, 2022.
- [17] “halo2 - the halo2 book.” <https://zcash.github.io/halo2/index.html>.
- [18] iden3, “Iden3/snarkjs: Zksnark implementation in javascript and wasm.” <https://github.com/iden3/snarkjs>.
- [19] G. Barthe, J. M. Crespo, and C. Kunz, “Relational verification using product programs,” in *International Symposium on Formal Methods*, pp. 200–214, Springer, 2011.
- [20] S. K. Lahiri and S. Qadeer, “Complexity and algorithms for monomial and clausal predicate abstraction,” in *International Conference on Automated Deduction*, pp. 214–229, Springer, 2009.
- [21] RISC Zero, “ZirGen: Risc zero circuit generator.” <https://github.com/risc0/zirgen>, 2024. Accessed: 2024-11-01.
- [22] ZoKrates Team, *ZoKrates: A Toolbox for zkSNARKs on Ethereum*, 2024. Official Documentation.
- [23] J. Ax, “The elementary theory of finite fields,” *Annals of Mathematics*, vol. 88, no. 2, pp. 239–271, 1968.
- [24] I. Dillig, T. Dillig, and A. Aiken, “Fluid updates: Beyond strong vs. weak updates,” in *Programming Languages and Systems: 19th European Symposium on Programming, ESOP 2010, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2010, Paphos, Cyprus, March 20-28, 2010. Proceedings 19*, pp. 246–266, Springer, 2010.
- [25] P. Cousot and R. Cousot, “Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fix-points,” in *Proceedings of the 4th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, pp. 238–252, 1977.
- [26] C. Flanagan and K. R. M. Leino, “Houdini, an annotation assistant for esc/java,” in *International Symposium of Formal Methods Europe*, pp. 500–517, Springer, 2001.
- [27] “Circom compiler.” <https://github.com/iden3/circom>.
- [28] L. de Moura and N. Bjørner, “Z3: An efficient smt solver,” in *Tools and Algorithms for the Construction and Analysis of Systems* (C. R. Ramakrishnan and J. Rehof, eds.), (Berlin, Heidelberg), pp. 337–340, Springer Berlin Heidelberg, 2008.
- [29] A. Coglio, E. McCarthy, E. Smith, C. Chin, P. Gaddamadugu, and M. Dellepere, “Compositional formal verification of zero-knowledge circuits.” Cryptology ePrint Archive, Paper 2023/1278, 2023.
- [30] C. Kwan, Q. Dao, and J. Thaler, “Verifying jolt zkVM lookup semantics.” Cryptology ePrint Archive, Paper 2024/1841, 2024.
- [31] M. Stronati, D. Firsov, A. Locascio, and B. Livshits, “Clap: a semantic-preserving optimizing edsl for plonkish proof systems,” 2024.

- [32] H. Chen, G. Li, M. Chen, R. Liu, and S. Gao, “Ac4: Algebraic computation checker for circuit constraints in zkps,” 2024.
- [33] R. Sharma, E. Schkufza, B. Churchill, and A. Aiken, “Data-driven equivalence checking,” *SIGPLAN Not.*, vol. 48, p. 391–406, Oct. 2013.
- [34] B. Churchill, O. Padon, R. Sharma, and A. Aiken, “Semantic program alignment for equivalence checking,” in *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2019, (New York, NY, USA), p. 1027–1040, Association for Computing Machinery, 2019.
- [35] M. Dahiya and S. Bansal, “Black-box equivalence checking across compiler optimizations,” in *Programming Languages and Systems* (B.-Y. E. Chang, ed.), (Cham), pp. 127–147, Springer International Publishing, 2017.
- [36] R. Dickerson, P. Mukherjee, and B. Delaware, “Kestrel: Relational verification using e-graphs for program alignment,” 2024.
- [37] J. Chen, J. Wei, Y. Feng, O. Bastani, and I. Dillig, “Relational verification using reinforcement learning,” *Proc. ACM Program. Lang.*, vol. 3, Oct. 2019.
- [38] S. Gulwani and A. Tiwari, “An abstract domain for analyzing heap-manipulating low-level software,” in *Proceedings of the 19th International Conference on Computer Aided Verification*, CAV’07, (Berlin, Heidelberg), p. 379–392, Springer-Verlag, 2007.
- [39] C. Flanagan and S. Qadeer, “Predicate abstraction for software verification,” *SIGPLAN Not.*, vol. 37, p. 191–202, Jan. 2002.
- [40] D. Gopan, T. Reps, and M. Sagiv, “A framework for numeric analysis of array operations,” *SIGPLAN Not.*, vol. 40, p. 338–350, Jan. 2005.
- [41] S. Gulwani, B. McCloskey, and A. Tiwari, “Lifting abstract interpreters to quantified logical domains,” *SIGPLAN Not.*, vol. 43, p. 235–246, Jan. 2008.
- [42] L. Kovács and A. Voronkov, “Finding loop invariants for programs over arrays using a theorem prover,” in *Fundamental Approaches to Software Engineering* (M. Chechik and M. Wirsing, eds.), (Berlin, Heidelberg), pp. 470–485, Springer Berlin Heidelberg, 2009.
- [43] I. Dillig, T. Dillig, and A. Aiken, “Symbolic heap abstraction with demand-driven axiomatization of memory invariants,” *SIGPLAN Not.*, vol. 45, p. 397–410, Oct. 2010.

10 Appendix

10.1 Join operation on Symbolic Stores

In this section we present and prove theorems about the join operators. First, we formally define the containment operator $\sqsubseteq$ between symbolic stores.

Definition 7. Let Σ and Σ' be symbolic stores. We write $\Sigma \sqsubseteq \Sigma'$ if and only if (1) If for every $(s, \varphi) \in \text{dom}(\Sigma)$ we have $\Sigma(s, \varphi) \sqsubseteq \Sigma'(s, \varphi)$ and (2) For any $(s', \varphi') \in \text{dom}(\Sigma')$ and for all $(s, \varphi) \in \text{dom}(\Sigma)$, if $s = s'$ and $\text{SAT}(\varphi = \varphi')$, then $\Sigma(s, \varphi) \sqsubseteq \Sigma'(s', \varphi')$.

Intuitively, the second condition asserts that if two entries could alias, then Σ' should allow the two entries to agree when concretely instantiated.

Lemma 1. *Let φ and φ' be symbolic expressions. Then $\varphi \sqsubseteq \varphi \sqcup \varphi'$ and $\varphi' \sqsubseteq \varphi \sqcup \varphi'$*

Proof. The proof follows immediately from the definition of $\sqsubseteq$ over symbolic expression when analyzing rules SYMEXP-1 and SYMEXP-2

Lemma 2. *For any symbolic stores Σ_1 and Σ_2 we have $\Sigma_1 \sqsubseteq \Sigma_1 \sqcup \Sigma_2$ and $\Sigma_2 \sqsubseteq \Sigma_1 \sqcup \Sigma_2$.*

Proof. We first show that for any $(v, \varphi) \in \text{dom}(\Sigma_1)$ that $\Sigma_1(v, \varphi) \sqsubseteq (\Sigma_1 \sqcup \Sigma_2)(v, \varphi)$. Take $(v, \varphi) \in \text{dom}(\Sigma_1)$. Then there are two possibilities:

- $(v, \varphi) \in \text{dom}(\Sigma_2)$: In this case, we know from the BOTH rule that $(\Sigma_1 \sqcup \Sigma_2)[(v, \varphi)] = \Sigma_1[(v, \varphi)] \sqcup \Sigma_2[(v, \varphi)]$. As such, we can conclude (using Lemma 1) that $\Sigma_1(v, \varphi) \sqsubseteq \Sigma_1[(v, \varphi)] \sqcup \Sigma_2[(v, \varphi)]$.
- $(v, \varphi) \notin \text{dom}(\Sigma_2)$: From the ONE rule, we know that $(\Sigma_1 \sqcup \Sigma_2)[(v, \varphi)] = \varphi'$ where φ' is a fresh variable. Since φ' is fresh, we have that $\Sigma_1(v, \varphi) \sqsubseteq \varphi'$ and hence $\Sigma_1(v, \varphi) \sqsubseteq (\Sigma_1 \sqcup \Sigma_2)[(v, \varphi)]$

We now show that for any $(s', \varphi') \in \text{dom}(\Sigma_1 \sqcup \Sigma_2)$ and for all $(s, \varphi) \in \text{dom}(\Sigma_1)$, if $s = s'$ and $\varphi \neq \varphi'$, then $\Sigma(s, \varphi) \sqsubseteq \Sigma'(s', \varphi')$. Take $(s', \varphi') \in \text{dom}(\Sigma_1 \sqcup \Sigma_2)$ and let us assume that $(s', \varphi') \notin \text{dom}(\Sigma_1)$. Then it must be the case that $(s', \varphi') \in \text{dom}(\Sigma_2)$. Thus, the ONE rule would set $\Sigma_1 \sqcup \Sigma_2[(s', \varphi')]$ to be a fresh variable. Thus for any $(s', \varphi') \in \Sigma_1$ it would follow that $\Sigma(s', \varphi') \sqsubseteq \Sigma_1 \sqcup \Sigma_2[(s', \varphi')]$.

10.2 Operational Semantics of Product Programs

In order to prove the soundness of our static analysis, we first describe the operational semantics of product programs. The states of our product program are referred to here as *concrete stores*, denoted by $\mathbb{C}$. Moreover, we continue with the assumption made in Section 5 where we assume all signals and data variables are modeled as flat arrays. As such, a $\mathbb{C}$ maps array-index pairs to values where the arrays. Likewise, template parameters are also modeled as arrays of length 1. As such, we say an *input* concrete state is a concrete state which does the following:

1. Maps all input witness signals to values.
2. Maps all template parameters to values.
3. Maps all constraint signals to values

The big step operational semantics of product programs are presented in Figure 25. They are judgments of the form $\mathbb{C} \vdash S \Downarrow \mathbb{C}', b$ where b indicates whether the constraints in S are satisfied. In the program's initial state, b is set to True. The judgment means that evaluating S given concrete store $\mathbb{C}$ results in a new store $\mathbb{C}'$ and conditional b' .

Definition 8 (Satisfying Input). *Given a template $\mathcal{T} = (V, \Gamma, \mathbb{F}_p, \mathcal{P}, \Sigma_\Phi, \Sigma_\omega)$, let $P_\otimes$ denote the corresponding product program. We say an input concrete store I is a satisfying input for $P_\otimes$ if and only if $I \vdash P_\otimes \Downarrow \mathbb{C}, \text{True}$*

$$\begin{array}{c}
\frac{\llbracket E_1 \rrbracket_{\mathbb{C}} \rightarrow c_1, \llbracket E_2 \rrbracket_{\mathbb{C}} \rightarrow c_2}{\mathbb{C} \vdash v[E_1] \leftarrow E_2 \Downarrow \mathbb{C}[(v, c_1) \rightarrow c_2], \text{True}} (\text{ARR-ASSIGN}) \\
\\
\frac{\llbracket E_1 \rrbracket_{\mathbb{C}} \rightarrow c_1, \llbracket E_2 \rrbracket_{\mathbb{C}} \rightarrow c_2, c_1 \neq c_2}{\mathbb{C} \vdash E_1 == E_2 \Downarrow \mathbb{C}, \text{False}} (\text{CONSTRAINT-FALSE}) \\
\\
\frac{\llbracket E_1 \rrbracket_{\mathbb{C}} \rightarrow c_1, \llbracket E_2 \rrbracket_{\mathbb{C}} \rightarrow c_2, c_1 = c_2}{\mathbb{C} \vdash E_1 == E_2 \Downarrow \mathbb{C}, b} (\text{CONSTRAINT-TRUE}) \\
\\
\frac{\mathbb{C} \vdash S_1 \Downarrow \mathbb{C}_1, b_1 \quad \mathbb{C}_1 \vdash S_2 \Downarrow \mathbb{C}_2, b_2}{\mathbb{C} \vdash S_1; S_2 \Downarrow \mathbb{C}_2, b_1 \wedge b_2} (\text{SEQUENCE}) \\
\\
\frac{\llbracket C \rrbracket_{\mathbb{C}} \neq 0 \quad \mathbb{C} \vdash S_1 \Downarrow \mathbb{C}', b}{\mathbb{C} \vdash \text{if}(C) S_1 \text{ else } S_2 \Downarrow \mathbb{C}', b} (\text{IF-TRUE}) \\
\\
\frac{\llbracket C \rrbracket_{\mathbb{C}} = 0 \quad \mathbb{C} \vdash S_2 \Downarrow \mathbb{C}', b}{\mathbb{C} \vdash \text{if}(C) S_1 \text{ else } S_2 \Downarrow \mathbb{C}', b} (\text{IF-FALSE}) \\
\\
\frac{\llbracket C \rrbracket_{\mathbb{C}} \neq 0 \quad \mathbb{C} \vdash S \Downarrow \mathbb{C}_1, b \quad \mathbb{C}_1 \vdash \text{while}(C) S \Downarrow \mathbb{C}', b'}{\mathbb{C} \vdash \text{while}(C) S \Downarrow \mathbb{C}', b \wedge b'} (\text{WHILE-TRUE}) \\
\\
\frac{\llbracket C \rrbracket_{\mathbb{C}} = 0}{\mathbb{C} \vdash \text{while}(C) S \Downarrow \mathbb{C}, \text{True}} (\text{WHILE-FALSE})
\end{array}$$

Fig. 25: Rules defining big step operational semantics of product programs

10.3 Soundness of Static Analysis

Completeness Assumption: In this section, we assume the product programs are such that in every instantiation every witness signal is written to for all inputs and for every constraint signal, its corresponding witness signal is written to. More precisely, given a product program $P_{\otimes}$ and a satisfying input I where $I \vdash P_{\otimes} : \mathbb{C}, \text{True}$. Then for every c and every pair of corresponding witness and constraint signals s_w and s_c , $(s_w, c) \in \text{dom}(\mathbb{C})$ if and only if $(s_c, c) \in \text{dom}(\mathbb{C})$.

Write-Once Assumption: We also assume that product programs are such that in every instantiation, each witness signal (or signal array element) is written exactly once.

Definition 9 (Interpretation of Symbolic Store). Let Σ be a symbolic store that corresponds to product program $P_{\otimes}$. We say an interpretation of Σ is a mapping $M = M_1 \cup M_2$ where M_1 assigns values to input signals of $P_{\otimes}$ and M_2 assigns fresh variables to values.

If φ is a symbolic expression and M an interpretation, then we write $M(\varphi)$ to denote the value produced when replacing all variables in φ with values assigned by M .

Definition 10 (Concretization of Symbolic Store). *Let Σ be a symbolic store and $M = M_1 \cup M_2$ be an interpretation of Σ . The concretization of Σ under M is the following set:*

$$M_1 \cup \bigcup_{((s, \varphi), \varphi') \in \Sigma} ((s, M(\varphi)), M(\varphi'))$$

We write $M(\Sigma)$ to denote the concretization of Σ .

We refer to $M(\sigma)$ as a *valid concretization* if for every pair of entries $((s, \varphi_1), \varphi'_1) \in \Sigma$ and $((s, \varphi_2), \varphi'_2) \in \Sigma$ such that $M(\varphi_1) = M(\varphi_2)$ we have $M(\varphi'_1) = M(\varphi'_2)$. Given a concrete store $\mathbb{C}$, we say that M is a valid concretization with respect to $\mathbb{C}$ if the above property holds for all $(s, c) \in \text{dom}(\mathbb{C})$ and use the notation $M_{\mathbb{C}}(\Sigma)$ to denote the restricted concretization.

Definition 11 (Agreement of Symbolic Store). *We say that a symbolic store Σ strongly agrees with a concrete store $\mathbb{C}$ (denoted by $\mathbb{C} \sim \Sigma$) if there is an interpretation $M = M_1 \cup M_2$ such that (1) M is a valid concretization with respect to $\mathbb{C}$ and (2) $M_{\mathbb{C}}(\Sigma) = \mathbb{C}$.*

We refer to such an M as an agreement witness between Σ and $\mathbb{C}$.

Lemma 3. *[Soundness of Expression Lookup] Let $\mathbb{C}$ be a concrete store, and Σ be a symbolic store such that $\mathbb{C} \sim \Sigma$. Let $M = M_1 \cup M_2$ be an agreement witness between $\mathbb{C}$ and Σ . Then for any expression E such that $\Sigma \vdash E : \varphi$, there is an interpretation $M' = M_1 \cup M'_2$ where $M_2 \subseteq M'_2$ and $M'(\varphi) = \llbracket E \rrbracket_{\mathbb{C}}$*

Proof. We prove this by structural induction on E .

- **Base Case** ($E = \alpha$ or $E = c$): If E is a template parameter α , then it must be the case that $M_1(\alpha) = \mathbb{C}(\alpha)$ as the symbolic store only tracks the possible values signals and data variables. Thus, $M' = M$. Likewise, constants are fixed in all interpretations so in both cases M remains unchanged.
- **Input Signals** ($E = s_{in}[E']$): Under the concrete store, this expression will evaluate to $s_{in}[\llbracket E' \rrbracket_{\mathbb{C}}]$. Our rule for this expression will first lookup E' and suppose that yields a symbolic expression φ . Applying our inductive hypothesis on E' yields a map $M' = M_1 \cup M'_2$ such that $M_2 \subseteq M'_2$ and $M'_2(\varphi) = \llbracket E' \rrbracket_{\mathbb{C}}$. Thus, $M'(s_{in}[E']) = s_{in}[\llbracket E' \rrbracket_{\mathbb{C}}]$ and so M' is a suitable extension.
- **Array Lookup** ($E = v[E']$). Our lookup rules will first lookup E' and suppose that yields a symbolic expression φ . From our inductive hypothesis there is an extension $M' = M_1 \cup M'_2$ such that $M_2 \subseteq M'_2$ and $M'(\varphi) = \llbracket E' \rrbracket_{\mathbb{C}}$. There are three cases two consider:

- $(v, \varphi) \in \text{dom}(\Sigma)$: Since M' is an extension of M it follows that $((v, \varphi), \varphi') \in M'$. As $M'(\varphi) = \llbracket E' \rrbracket_{\mathbb{C}}$ and M' is a valid concretization of Σ , it follows that $M'(\varphi') = \llbracket v[E'] \rrbracket$. Hence M' is a suitable extension.
 - $(v, \varphi) \notin \text{dom}(\Sigma)$ and v is not an input signal: In this case, the ARR-2 is invoked and the helper rule UNKNOWN is invoked to generate a fresh variable ς . A suitable extension is $M' = M_1 \cup M_2''$ where $M_2'' = M_2'[\varsigma \leftarrow \llbracket v[E'] \rrbracket]$. This is because ς is fresh so its assignment does not affect the concrete store produced under M' .
 - $(v, \varphi) \notin \text{dom}(\Sigma)$ and v is an input signal: In this case, the ARR-3 rule is invoked and it produces the symbolic expression $v[\varphi]$. Since v is an input signal, we have $M'(v[\varphi]) = M_1(v[\llbracket E' \rrbracket_{\mathbb{C}}]) = \mathbb{C}(v, \llbracket E' \rrbracket_{\mathbb{C}})$.
- **Arithmetic** ($E = f(E_1, \dots, E_n)$) Under the concrete store this expression evaluates to $f(\llbracket E_1 \rrbracket_{\mathbb{C}}, \dots, \llbracket E_n \rrbracket_{\mathbb{C}})$. Our arithmetic rule looks up each $E_1, \dots, E_n$ yielding symbolic expressions $\varphi_1, \dots, \varphi_n$ and then returns the expression $f(\varphi_1, \dots, \varphi_n)$. As such, if we apply our inductive hypothesis successively to each of $E_1 \dots E_n$ then it follows that there is an extension M' such that $M'(\varphi_i) = \llbracket E_i \rrbracket_{\mathbb{C}}$ for each $1 \leq i \leq n$. Under that extension $M'(f(E_1, \dots, E_n)) = f(\llbracket E_1 \rrbracket_{\mathbb{C}}, \dots, \llbracket E_n \rrbracket_{\mathbb{C}}) = \llbracket f(E_1, \dots, E_n) \rrbracket_{\mathbb{C}}$

Observe that if M is an agreement witness between $\mathbb{C}$ and Σ and if M' is an extension of M , then M' is also an agreement witness.

Lemma 4. *Suppose we have a concrete store $\mathbb{C}$ and a symbolic store Σ such that $\mathbb{C} \sim \Sigma$. Suppose $\Sigma \sqsubseteq \Sigma'$. Then, $\mathbb{C} \sim \Sigma'$.*

Proof. For simplicity, let us assume that every entry $(s, \varphi) \in \text{dom}(\Sigma)$ is such that s is a variable array in $\mathbb{C}$. That is because all other entries are not relevant with respect to the agreement witness.

Now, let M be an agreement witness between $\mathbb{C}$ and Σ . We claim there is an extension M' of M such that M' is an agreement witness between $\mathbb{C}$ and Σ' . We build such an M' incrementally over each entry of Σ' and set M to M' at the end. Thus, $M = M'$ at the beginning and end of each entry. There are two cases we need to consider among the entries:

- For any $(v, \varphi) \in \text{dom}(\Sigma') \cap \text{dom}(\Sigma)$, we have $\Sigma[(v, \varphi)] \sqsubseteq \Sigma'[(v, \varphi)]$. From the definition of $\sqsubseteq$ between symbolic expressions there are two sub cases. If $\Sigma[(v, \varphi)] = \Sigma'[(v, \varphi)]$, then M' does not need to change for this entry. If not, then $\Sigma'[(v, \varphi)]$ is a fresh unknown ς and so M' can assign ς to $M(\Sigma(v, \varphi))$. Thus, M' yields the same concrete store with respect to all entries in Σ .
- Now consider any $(v, \varphi') \in \text{dom}(\Sigma') - \text{dom}(\Sigma)$. Again for simplicity, let us assume v is a variable in $\mathbb{C}$. As such, there is an entry $(v, \varphi) \in \text{dom}(\Sigma)$ but $\varphi \neq \varphi'$. Since $\Sigma \sqsubseteq \Sigma'$, it follows from Definition 7 that $\varphi \sqsubseteq \varphi'$ and $\Sigma(v, \varphi) \sqsubseteq \Sigma'(v, \varphi')$. Since $\varphi \neq \varphi'$ it follows that φ' is a fresh variable. As such, we extend M' to assign φ' to $M(\varphi)$ so that the two indexes agree in the concretization of Σ' . To ensure M' is a valid concretization we need to make sure that $M'(\Sigma(v, \varphi')) = M(\Sigma(v, \varphi))$ in the concretization. Since $\Sigma(v, \varphi) \sqsubseteq \Sigma'(v, \varphi')$ we can follow the reasoning in the first case to obtain the desired M' .

Lemma 5. *[Soundness of CONSTRUCTSYMBOLICSTORE] Given a template $\mathcal{T} = (V, \Gamma, \mathbb{F}_p, \mathcal{P}, \Sigma_\Phi, \Sigma_\omega)$, let $P_\otimes$ denote the corresponding product program. Let Σ be the result of calling CONSTRUCTSYMBOLICSTORE on $P_\otimes$. Then for any satisfying input I , if $\mathbb{C}$ is the resulting concrete store produced by running $P_\otimes$ on I , then $\mathbb{C} \sim \Sigma$.*

Proof. Let S be the body of $P_\otimes$. We will show that if $\mathbb{C} \sim \Sigma$, then if $\Sigma \vdash S : \Sigma'$ and $\mathbb{C} \vdash S \Downarrow \mathbb{C}', \text{True}$, then $\mathbb{C}' \sim \Sigma$. We do so via structural induction on S .

- **Base Case 1** $(s_w[E_1] \leftarrow E_2)$: For this statement, the analysis will invoke the ASGN-SIG rule which first looks up E_1 and E_2 , returning symbolic expressions φ_1 and φ_2 . It then maps (s_w, φ_1) to φ_2 . During concrete execution, the expression E_1 is evaluated (yielding a value c) along with E_2 (yielding a value c') and the concrete store is updated to map (s_w, c) to c' . Now, since $\mathbb{C} \sim \Sigma$ it follows that there is an agreement witness M . From Lemma 5 we know there is an extension M' of M such that $M'(E_1) = \llbracket E_1 \rrbracket_{\mathbb{C}}$ and $M'(E_2) = \llbracket E_2 \rrbracket_{\mathbb{C}}$. We claim that: M' is an agreement witness between $\mathbb{C}'$ and Σ' and to do we show that (1) $M'_{\mathbb{C}'}(\Sigma')$ is a valid concretization (2) $M'^{\mathbb{C}'}(\Sigma') = \mathbb{C}'$ and we start by proving (1):

- Suppose $M'_{\mathbb{C}'}(\Sigma')$ is not a valid concretization. Then there must be two entries $((s, \varphi_1), \varphi_2)$ and $((s, \varphi'_1), \varphi'_2) \in \Sigma$ such that $M'(\varphi_1) = M'(\varphi'_1) = c$ but $M'(\varphi_2) \neq M'(\varphi'_2)$. We know that one of the entries must be the new entry $((s_w, \varphi_1), \varphi_2)$ and the other must be some old entry $((s_w, \varphi'_1), \varphi'_2)$ as otherwise, $M'_{\mathbb{C}}(\Sigma)$ would not have been a valid concretization. Since $((s_w, \varphi'_1), \varphi'_2) \in \Sigma$ and $M'_{\mathbb{C}}(\Sigma) = \mathbb{C}$ it follows that $((s_w, c), M'(\varphi'_2)) \in \mathbb{C}$. But this means the entry (s_w, c) was already written to during the execution of the product program, violating our assumption that signal array elements are only written once during program execution.

We now show that $M'_{\mathbb{C}'}(\Sigma') = \mathbb{C}'$. First, observe that the new entry $((s_w, c), c') \in M'(\Sigma')$. Now, we show that all other entries in $\mathbb{C}'$ are precisely the other entries in $M'_{\mathbb{C}'}(\Sigma')$. There are two cases to consider:

1. Every $(v, c') \in \text{dom}(\mathbb{C})$ such that $v \neq s_w$ will also be in $\text{dom}(\mathbb{C}')$. Likewise the corresponding entry $(v, \varphi) \in \text{dom}(\Sigma)$ remains unchanged. Thus, the concretization of Σ' under M' includes all such entries.
 2. Take an entry $(s_w, c') \in \text{dom}(\mathbb{C}')$ such that $c' \neq c$ and assume for reductio that $(s_w, c') \notin \text{dom}(M'_{\mathbb{C}'}(\Sigma'))$. Since $c \neq c'$, it must be that $(s_w, c) \in \text{dom}(M'_{\mathbb{C}'}(\Sigma))$ and since the only entry added/changed was (s_w, φ_1) , it must also be the case that $(s, \varphi_1) \in \text{dom}(\Sigma)$. Thus, $c' = M'_{\mathbb{C}'}(\Sigma')(\varphi_1) = c'$. But since $c \neq c'$ we have a contradiction and so $(s_w, c') \in \text{dom}(M'_{\mathbb{C}'}(\Sigma'))$.
- **Base Case 2** $(v[E_1] \leftarrow E_2)$: On this type of statement, the analysis will invoke the ASGN-VAR rule which looks up E_1 and E_2 , yielding symbolic expressions φ_1 and φ_2 . It then updates the store by adding the entry $((v, \varphi_1), \varphi_2)$. However, since data variables entries can alias, it also finds all entries $(v, \varphi') \in \Sigma$ such that $\text{SAT}(\varphi' = \varphi_1)$ and for each such entry, it havoc's the value in the symbolic store. It does so by creating a fresh variable ς' for each entry and updates the store so that $\Sigma[(v, \varphi') \leftarrow \varsigma']$. Under the concrete execution, the entry $((v, \llbracket E_1 \rrbracket_{\mathbb{C}}), \llbracket E_2 \rrbracket_{\mathbb{C}})$ is added to $\mathbb{C}$ if $(v, \llbracket E_1 \rrbracket_{\mathbb{C}}) \notin \text{dom}(\mathbb{C})$ or the ex-

isting entry is set to $\llbracket E_2 \rrbracket_{\mathbb{C}}$. Like in the proof of Base Case 1, we know there is an agreement witness M between $\mathbb{C}$ and Σ and by Lemma 2, we know there is an extension M' of M such that $M'(E_1) = \llbracket E_1 \rrbracket_{\mathbb{C}}$ and $M'(E_2) = \llbracket E_2 \rrbracket_{\mathbb{C}}$. Let $\mathcal{S}$ denote the set of entries (v, φ') such that $SAT(\varphi' = \varphi)$ and let $\mathcal{A}$ denote the set of entries (v, φ') such that $M'(\varphi') = M'(\varphi_1)$. Intuitively, the former is the set of entries that *may* alias and the latter is the set which *actually* alias. Clearly $\mathcal{A} \subseteq \mathcal{S}$. Now, we define an M'' that agrees with M' on all entries except for $(v, \varphi') \in \mathcal{A}$. In that case, we have M'' assign the fresh variable to $\llbracket E_2 \rrbracket_{\mathbb{C}}$. Similarly, for every entry $(v, \varphi') \in \mathcal{S} - \mathcal{A}$ we have M'' map the fresh variable to it's old value. We claim M'' is an agreement witness between $\mathbb{C}'$ and Σ' . To see why, we consider two cases:

1. We observe that for every $(v', c) \in \mathbb{C}$ such that $v' \neq v$, the entry still remains in $\mathbb{C}'$. Likewise, its corresponding entry remains in Σ' as the ASGN-VAR rule only updates entries that correspond to the same data array. Hence all such entries are present in $M'_{\mathbb{C}'}(\Sigma')$ and vice versa.
 2. Every entry $(v, c') \in \mathbb{C}$ where $c' \neq c$ either corresponds to an entry $(v, \varphi') \in \Sigma$ such that $UNSAT(\varphi' = \varphi_1)$ or $SAT(\varphi' = \varphi_1)$. In the former case, the corresponding symbolic store entry is not updated and so the entry is present in $M'_{\mathbb{C}'}(\Sigma'')$ and vice versa. In the latter case there are two sub-possibilities:
 - $(v, \varphi') \in \mathcal{A}$. In this case, $M''(\varphi') = c$ and $c' = c$, and so (v, c') is set to $\llbracket E_2 \rrbracket_{\mathbb{C}}$ in $\mathbb{C}'$. As described above, in this case, M'' sets the fresh variable (created by havocing the existing entry) to $\llbracket E_2 \rrbracket_{\mathbb{C}}$ and so the entry $((v, c), \llbracket E_2 \rrbracket_{\mathbb{C}}) \in M'_{\mathbb{C}'}(\Sigma')$.
 - $(v, \varphi') \in \mathcal{S} - \mathcal{A}$. In this case, $M''(\varphi') = c' \neq c$. Thus, the entry corresponding to (v, c') is unchanged in $\mathbb{C}'$. As described above, in this case, M'' sets the fresh variable created by havocing the existing entry) to the old entry. Hence the old entry remains in $M'_{\mathbb{C}'}(\Sigma')$.
- **Base Case 3** ($s_c[E_1] == E_2$): In this case the analysis will invoke the CSTR rule which looks up E_1 and E_2 , yielding symbolic expressions φ_1 and φ_2 . It then updates the store by adding the entry $((s_c, \varphi_1), \varphi_2)$. The concrete store remains unchanged but since it corresponds to a satisfying input we know that the entry $((s_c, \llbracket E_1 \rrbracket_{\mathbb{C}}), \llbracket E_2 \rrbracket_{\mathbb{C}}) \in \mathbb{C}$. Since $\mathbb{C} \sim \Sigma$ it follows that there is an agreement witness between $\mathbb{C}$ and Σ , and from Lemma 5 we know there is an extension M' of M such that $M'(E_1) = \llbracket E_1 \rrbracket_{\mathbb{C}}$ and $M'(E_2) = \llbracket E_2 \rrbracket_{\mathbb{C}}$. We claim that: (1) $M'_{\mathbb{C}'}(\Sigma')$ is a valid concretization and (2) $M'_{\mathbb{C}'}(\Sigma') = \mathbb{C}'$.
- To see why $M'_{\mathbb{C}'}(\Sigma')$ is a valid concretization, suppose it was not. Since $M'_{\mathbb{C}}$ was a valid concretization for Σ it must be the case that there is some entry $((s_c, \varphi'_1), \varphi'_2)$ that conflicts with the new entry $((s_c, \varphi_1), \varphi_2)$. In particular, this means $M'(\varphi_1) = M'(\varphi'_1) = \llbracket E_1 \rrbracket_{\mathbb{C}}$ and $M'(\varphi_2) \neq M'(\varphi'_2)$. However, since $M'_{\mathbb{C}}(\Sigma)$ is a valid concretization and $M'_{\mathbb{C}}(\Sigma) = \mathbb{C}$, it means that $((s_c, \llbracket E_1 \rrbracket_{\mathbb{C}}), M'(\varphi'_2)) \in \mathbb{C}$. But this would mean the constraint is not satisfied, contradicting the fact that the input is satisfying. Thus, $M'_{\mathbb{C}'}(\Sigma')$ must be a valid concretization.
- Finally, $M'_{\mathbb{C}'}(\Sigma') = \mathbb{C}'$ follows from the fact that (1) $\mathbb{C}' = \mathbb{C}$ (2) $((s_c, \llbracket E_1 \rrbracket_{\mathbb{C}}), \llbracket E_2 \rrbracket_{\mathbb{C}}) \in M'(\Sigma')$, and (3) all other entries of Σ' remain unchanged.
- **Seq** ($S_1; S_2$): Suppose we have $\mathbb{C} \vdash S_1 \Downarrow \mathbb{C}_1, \text{True}$ and $\mathbb{C}_1 \vdash S_2 \Downarrow \mathbb{C}_2, \text{True}$. Likewise suppose in the analysis we have $\Sigma \vdash S_1 : \Sigma_1$ and $\Sigma_1 \vdash S_2 : \Sigma_2$.

- Since $\mathbb{C} \sim \Sigma$, we can conclude from our inductive hypothesis that $\mathbb{C}_1 \sim \Sigma_1$. From there we can similarly conclude that $\mathbb{C}_2 \sim \Sigma_2$.
- **If:** Suppose have $\Sigma \vdash S_1 : \Sigma_1$ and $\Sigma \vdash S_2 : \Sigma_2$. Then the resulting store is $\Sigma_1 \sqcup \Sigma_2$. In the concrete execution there are two possibilities:
 - $\llbracket C \rrbracket_{\mathbb{C}} = \text{True}$ and so the resulting store is simply $\mathbb{C} \vdash S_1 \Downarrow \mathbb{C}_1, \text{True}$. Since $\mathbb{C} \sim \Sigma$ it follows that $\mathbb{C}_1 \sim \Sigma_1$ and since $\Sigma_1 \sqsubseteq \Sigma_1 \sqcup \Sigma_2$ it follows from Lemma 4 that $\mathbb{C}_1 \sim \Sigma_1 \sqcup \Sigma_2$.
 - $\llbracket C \rrbracket_{\mathbb{C}} = \text{False}$. The reasoning in this case is nearly identical as in the case above.
 - **While:** For while loops, the rule asserts that the symbolic store must be inductive with respect to the loop body. So suppose we have a symbolic store Σ such that $\Sigma \vdash S : \Sigma'$ and $\Sigma' \sqsubseteq \Sigma$. There are two possibilities in the concrete store:
 - $\llbracket C \rrbracket = \text{True}$ and $\mathbb{C} \vdash S \Downarrow \mathbb{C}'$. From our inductive hypothesis it follows that $\mathbb{C}' \sim \Sigma'$. From Lemma 4, since $\Sigma' \sqsubseteq \Sigma$ we have $\mathbb{C}' \sim \Sigma$.
 - $\llbracket C \rrbracket = \text{False}$ and $\mathbb{C} \vdash \text{while}(C) S \Downarrow \mathbb{C}, \text{True}$. Since the concrete store doesn't change and the symbolic store doesn't change, we trivially have $\mathbb{C}' = \Sigma'$.

Definition 12 (Signal Equivalence). *Given a template $\mathcal{T} = (V, \Gamma, \mathbb{F}_p, \mathcal{P}, \Sigma_\Phi, \Sigma_\omega)$ let $P_\otimes$ denote the corresponding product program. Given a signal $s \in \Gamma$, we say that its witness generation component s_w and constraint component s_c are equivalent (written $s_w \equiv s_c$) if and only if for every satisfying input I , if $\mathbb{C}$ is the concrete store produced by running $P_\otimes$ on I , then $\mathbb{C}(s_c) = \mathbb{C}(s_w)$.*

Lemma 6. *Suppose $\Sigma \vdash s_c \sim s_w$. Then for any agreement witness M of Σ , $M(\Sigma)[s_w] = M(\Sigma)[s_c]$.*

Proof. Take any $((s_w, c), c') \in M(\Sigma)$. Since M is a concretization of Σ , there is some $((s_w, \varphi_1), \varphi_2) \in \Sigma$ such that $M(\varphi_1) = c$ and $M(\varphi_2) = c'$. Since $\Sigma \vdash s_c \sim s_w$ there is some $((s_c, \varphi'_1), \varphi'_2) \in \Sigma$ such that $\varphi_1 \equiv \varphi'_1$ and $\varphi_2 \equiv \varphi'_2$. Hence, $M(\varphi'_1) = c$ and $M(\varphi'_2) = c'$ and so $((s_c, c), c') \in M(\Sigma)$. As the entry was arbitrarily chosen, it follows that $M(\Sigma)[s_w] = M(\Sigma)[s_c]$.

Lemma 7 (Soundness of ISEQUIVALENT). *Given a template $\mathcal{T} = (V, \Gamma, \mathbb{F}_p, \mathcal{P}, \Sigma_\Phi, \Sigma_\omega)$ let $\Sigma = \text{CONSTRUCTSYMBOLICSTORE}(P_\otimes)$. For any pair of signals s_c, s_w if $\Sigma \vdash s_c \sim s_w$, then $s_c \equiv s_w$.*

Proof. Let I be a satisfying input to $P_\otimes$ and let $\mathbb{C}$ be the resulting concrete store. From Lemma 5 we have $\mathbb{C} \sim \Sigma$ and so there is an agreement witness M such that $\mathbb{C} = M_{\mathbb{C}}(\Sigma)$. From Lemma 6, we have $M_{\mathbb{C}}(\Sigma)[s_w] = M_{\mathbb{C}}(\Sigma)[s_c]$, and so it follows that $\mathbb{C}(s_w) = \mathbb{C}(s_c)$.

10.4 Example of Verification Failure


```

1 template verify_ec_compression(n, k) {
2   signal input uncompressed[2][k];
3   signal input compressed[33];
4
5   // Get the bit string of the smallest register \
6   // Make sure the least significant bit's evenness matches the evenness
7   // specified by the first byte in the compressed version
8   component num2bits = Num2Bits(n);
9   // Note, circom-ecdsa uses little endian, so we check the 0th register of the y value
10  num2bits.in <== uncompressed[1][0];
11  compressed[0] == num2bits.out[0] + 2;
12
13  // Make sure the compressed and uncompressed x coordinates represent
14  // the same number \
15  // 'l_bytes' is an algebraic expression for the bytes of each register
16  var l_bytes[k];
17  for (var i = 1; i < 33; i++) {
18    var j = i - 1; // ignores the first byte specifying the compressed y coordinate
19    l_bytes[j \ 8] += compressed[33-i] * (256 ** (j % 8));
20  }
21
22  uncompressed[0] == l_bytes;
23 }

```

Fig. 27: Template that is deterministic but incorrect

Figure 23 shows that ZEQUAL fails to verify in over half of the cases, primarily due to insufficient loop invariants. To illustrate a common pattern, consider the template shown in Figure 26, which copies entries from the array *in* to the array *out*. Instead of performing this operation in a single loop, the template uses two loops: the first loop assigns values to *out* in the witness generator, while the second loop constrains the values of *out*.

```

1 template TwoLoops(n) {
2   signal input in[n];
3   signal output out[n];
4   var i;
5   for(i = 0; i < n; i++) {
6     out[i] <-- in[i];
7   }
8
9   for(i = 0; i < n; i++) {
10    out[i] == in[i];
11  }
12 }

```

Fig. 26: Imprecise loop invariant example.

In this example, the predicates produced by the CONJECTUREPREDICATES procedure are insufficient because neither loop alone can prove the equivalence of *out*. Instead, more specific information about the relationship between *out* and *in* is required. For the second loop on line 9, the predicate

$$\forall x. (x \geq 0 \wedge x < n) \rightarrow out_w[x] = in_w[x]$$

is necessary to capture the assignments made in the first loop. Similarly, the first loop on line 5 requires the predicate

$$\forall x. (x \geq 0 \wedge x < i) \rightarrow out_w[x] = in_w[x]$$

to provide information needed for the verification of the second loop.

10.5 True Positive Example

In this section, we present an example of a circuit template that is deterministic but still incorrect. The circuit, shown in Figure 27, is designed to vali-

```

1 template EpochTreeLeaf(FIELD_COUNT) {
2   signal input epoch_key;
3   signal input data[FIELD_COUNT];
4   signal output out;
5
6   signal hasher[FIELD_COUNT];
7   component poseidons[FIELD_COUNT];
8
9   for (var x = 0; x < FIELD_COUNT; x++) {
10    poseidons[x] = Poseidon(2);
11    if (x == 0) {
12      poseidons[x].inputs[0] <== epoch_key;
13      poseidons[x].inputs[1] <== data[x];
14    } else {
15      poseidons[x].inputs[0] <== hasher[x-1];
16      poseidons[x].inputs[1] <== data[x];
17    }
18    hasher[x] <== poseidons[x].out;
19  }
20 }
21
22 out <== hasher[FIELD_COUNT-1];
23 }

```

Fig. 28: Example of a template that uses arrays to accumulate a value

date whether the input compressed point corresponds to a given uncompressed elliptic curve point. This template has no outputs, and thus cannot exhibit non-determinism. However, it violates consistency because it accepts inputs that are not produced by the witness generator. Specifically under certain instantiations, the invocation of $\text{Num2Bits}(k)$ at line 8 may generate a binary representation that can be manipulated by an attacker to bypass the constraint on line 10. As a result, the inputs may not adhere to the expected relationship between a compressed and uncompressed point.

10.6 Invariant Inference Example with Arrays

In this section, we present an example of a loop invariant generated by ZEQUAL for a common scenario where the input to one component of an array was generated by the output of a previous component. The circuit, shown in Figure 28, was taken from the Unirep project in our benchmark set and computes the hash of a given list by iteratively hashing the next element of the list together with the hash of the sub-list. To prove the consistency of this example, the following loop invariant is generated by ZEQUAL for the loop given on lines 9 to 20:

$$\begin{aligned}
&\forall y. \text{epoch_key}.c = \text{epoch_key}.w \wedge \\
&\quad x.c = x.w \wedge \\
&\quad \text{FIELD_COUNT}.c = \text{FIELD_COUNT}.w \wedge \\
&\quad (0 \leq y < \text{FIELD_COUNT}) \rightarrow (\text{data}[y].c = \text{data}[y].w) \wedge \\
&\quad (y < \text{FIELD_COUNT} \wedge 0 \leq y < x) \rightarrow (\text{hasher}[y].c = \text{hasher}[y].w)
\end{aligned}$$